PHP Gallery

Complete Product Documentation

Installation, administration, architecture, security, maintenance, and development reference

Application version:	0.97
Manual edition:	7 September 2026
Document format:	A4, indexed, linked PDF
Project author:	Rudolf Klusal
License:	MIT License

Guide to creating, organizing, presenting, protecting, sharing, and preserving a photographic collection with PHP Gallery version 0.97.

Contents

1 Purpose and reading guide

PHP Gallery is a self-hosted photo gallery. The photographs remain on storage you control, while the application adds the catalog, navigation, access rules, thumbnails, maps, search, downloads, and administrative tools around them. A single installation can be used for a public portfolio, a family archive, client delivery, event photographs, travel collections, or a large personal library.

The manual is organized around work that an owner actually performs. The early sections cover installation and the first gallery; the middle of the manual deals with publishing, media, privacy, presentation, and maintenance. Architecture and developer material is kept near the end so it does not interrupt ordinary administration.

For a new installation, begin with Section 5. Collection planning is covered in Section 6; routine gallery and photo editing is in Sections 7 and 9. Section 10 is useful before publishing because it compares several common audience types. The contents and index are intended for jumping directly to a setting or workflow when the site is already in use.

1.1 How to use this manual

Admin names such as **Theme** and **System Health** are shown in bold. Monospaced text denotes filenames, setting names, values, or commands. Procedures are written as ordered lists where sequence matters; checklists and reference material use bullets or tables. Technical implementation notes are included only where they explain a visible behavior, a recovery decision, or a maintenance requirement.

1.2 Further project references

Developer and hosting references remain separate from this owner manual. The References section lists the repository documents used for architecture, database, source ownership, testing, and contribution rules [1, 2, 3, 4, 6, 7].[1]

1.3 Terminology

Gallery A named collection of photographs. A gallery can contain photographs and smaller galleries.

Photograph or image One item in a gallery, together with its title, description, tags, visibility, and available camera information.

Thumbnail A smaller copy created for quick display in gallery grids, search results, covers, and previews.

Visitor A person viewing the public side of the site.

Administrator A trusted person who signs in to create and manage content and settings.

Selected gallery The gallery currently open on the screen.

Gallery branch A gallery together with every smaller gallery contained beneath it.

[1] Those files are the better source when changing code, inspecting schema details, or preparing a release.

2 Product overview

PHP Gallery turns a folder collection into a browsable photographic website. Visitors see covers, titles, descriptions, photo grids, full-screen viewing, maps, tags, search, and downloads according to the choices made by the owner. Administrators receive private tools for adding photographs, arranging collections, controlling privacy, improving descriptions, and keeping the installation healthy.

Ordinary PHP/MySQL web hosting is the primary deployment target. The owner keeps the source photographs in recognizable folders, while the database supplies the catalog and application state needed for browsing.[2]

2.1 Core capabilities

- Create galleries inside other galleries, for example *2026 / Italy / Rome* or *Clients / Northwind / Final selection*.
- Give every gallery a title, description, date range, cover, public address, privacy choice, and individual appearance.
- Upload photographs through the browser or discover files copied into gallery folders.
- Add titles, stories, tags, visibility, ordering, dates, and location information to photographs.
- Let visitors search, browse tags, view photographs full screen, follow maps, vote, and download permitted collections.
- Protect family, client, or sensitive collections with access controls and share methods.
- Find likely duplicate photographs and review them with direct links to both locations.
- Keep the installation current through guided updates, health checks, backups, and maintenance tools.
- Automate scoped uploads, exchange galleries with compatible installations, and prepare selected-photo or gallery downloads.
- Generate optional AI metadata and translation drafts, while keeping every generated value reviewable before publication.
- Publish crawler-friendly robots and image-sitemap output for the public content that visitors can actually access.

2.2 Filesystem and database partnership

PHP Gallery stores persistent data in both the filesystem and the database. Gallery directories hold the source photographs in a recognizable tree. The database stores the information that does not belong in the file itself: titles, descriptions, tags, ordering, access settings, votes, dates, generated-media records, and other application state.

Neither side is a complete backup on its own. Restoring only the database leaves catalog entries without their source files; restoring only the gallery tree loses the administrative and presentation state associated with those files. Backup and recovery should therefore treat the database, gallery directories, and local configuration as one set. Section 11 returns to this in more detail.

[2]The project overview provides the short installation and feature summary [1].

3 Requirements and environment

Use this section when choosing a hosting plan or checking an existing account. The requirements table is also suitable for sending directly to a hosting provider.

3.1 Minimum runtime

Component	Requirement
PHP	Version 8.1 or newer.
Database	MySQL 5.7 or newer, or MariaDB 10.2 or newer.
PDO	PDO MySQL extension for application queries and migrations.
Image processing	GD for common thumbnail generation. Additional local image tooling can support specialized formats.
Archives	ZipArchive for package, download, and transfer workflows that produce or consume ZIP data.
Filesystem	Writable application data, gallery, cache, and generated-media locations.
Web server	Apache-style rewriting is recommended for clean URLs; query-string routing supports compatible environments without rewrites.

Outbound HTTPS access supports application updates and remote release metadata. Local operation and manually deployed releases can function with a more restricted outbound policy.[3]

3.2 Permissions

The web process needs read access to runtime code and write access to designated mutable locations. Keep runtime code read-only where hosting permits it, except during a controlled updater operation. Gallery source files deserve backup coverage equal to the database because the complete application state spans both domains.

Important. Create backups of the database, gallery files, and installation configuration together. A database-only backup preserves metadata while omitting the authoritative media tree.

[3] Network policy should follow the installation owner's update and privacy requirements. Update code validates packages and preserves recovery data according to the local updater implementation.

4 Installation and initial setup

4.1 Browser installation

4.1.1 Installation sequence

The browser installer gathers database settings, initializes the schema through migrations, creates the first administrator, prepares writable locations, and writes local configuration. A fresh request redirects to installation when required configuration is absent.[4]

Typical steps are:

1. Create an empty MySQL or MariaDB database using the hosting control panel.
2. Upload the application package to the intended web directory.
3. Open the site and follow the installer.
4. Enter database connection values and the initial administrator credentials.
5. Confirm writable directories and extension checks.
6. Complete installation and enter the Admin area.

4.2 Command-line setup

4.2.1 Local commands

The repository provides plain PHP scripts for migration and administrator creation:

```
php scripts/migrate.php
php scripts/create_admin.php <username> <password>
```

Command-line setup follows the same persistent schema model as browser setup. Review `config.example.php`, create the local configuration, provision the database, run migrations, and create the first administrator.

4.3 First administrative review

After installation, review:

- Site name, language, theme, page width, and public rendering preferences.
- Gallery root permissions and discovery results.
- Thumbnail formats, dimensions, quality, and maintenance state.
- Access, password-reset, email, telemetry, update-channel, and backup preferences.
- Rewrite compatibility and canonical public URLs.
- System Health diagnostics and pending migrations.

[4] Installation state and generated configuration should receive the same protection as credentials. Source archives should continue excluding the local `config.php`.

4.4 Clean URLs and rewrite compatibility

PHP Gallery supports clean public paths when the hosting environment applies the project's rewrite rules. Clean URLs are a presentation and routing preference, not a requirement for the gallery data model. Query-string routes remain the compatibility path and continue to work when rewriting is disabled or cannot be verified reliably.

The URL rewriting setting controls which form PHP Gallery emits in generated links. When rewriting is off, helpers use query-style addresses for galleries, photographs, thumbnails, and other application routes instead of assuming that the web server will translate a clean path. Slugs, gallery folders, database rows, and source photographs do not need to be renamed when this setting changes.

If clean paths fail on a new host, first disable URL rewriting in Admin and confirm that the query-string form, such as `?page=gallery`, works. On Apache or LiteSpeed, then verify that the project's `.htaccess` is honored and that rewrite support is available. On other web servers, keep the compatibility mode unless equivalent routing rules have been configured.

5 Your first hour with PHP Gallery

For the first session, keep the scope small: create one gallery, add a few photographs, and open the result from a browser that is not signed in as Admin. Theme work and deeper organization can wait until that basic path is known to work.

5.1 Open the Admin area

Choose **Admin login** from the public header and enter the account created during installation. The dashboard is the starting point for everyday ownership. It summarizes galleries, photos, system state, and available maintenance actions.[5]

Spend a few minutes identifying these areas:

- **Galleries**, where collections are created, discovered, edited, and arranged;
- **Theme**, where the public site's appearance and presentation defaults are selected;
- **Tags**, where reusable subjects and categories are maintained;
- **System Health**, where migrations, thumbnails, storage, and consistency are reviewed;
- **Account**, where credentials, recovery email, and mail delivery are managed.

5.2 Create a first gallery

Open gallery administration and create a gallery with a short title such as *Summer Walk*. Review the proposed public path and folder name, then add one sentence describing what the collection contains.

For this first check, make the gallery public and leave advanced branding or inherited overrides at their defaults. Add three to ten photographs. That is enough to verify ordering, thumbnail generation, titles, and the lightbox without turning the exercise into a large import.

5.3 Preview as a visitor

Use the public-gallery link or visitor-preview control. For a realistic check, open the public address in a private browser window where the Admin session is absent. Confirm that the title, description, photographs, and navigation appear as intended. Open a photograph, move forward and backward in the viewer, and return to the gallery.

Administrator views can contain edit controls and additional assets. Anonymous preview gives the clearest picture of the experience received by family, clients, or the public.[6]

5.4 Complete the first-hour checklist

1. Set the public site name and language.

[5] The exact dashboard contents depend on enabled features, completed migrations, and development settings. A healthy installation can still display informational notices that invite an administrator to complete optional work.

[6] Protected galleries and age-restricted content deserve a separate anonymous test because an active administrator session already has broader access.

2. Create one gallery with a title and description.
3. Upload a small group of photographs.
4. Add a meaningful title to at least one photograph.
5. Reorder two photographs and confirm the new order publicly.
6. Open the gallery as an anonymous visitor.
7. Check the gallery on a phone-sized screen.
8. Confirm that System Health reports no urgent migration or permission problem.
9. Create a coordinated backup after the initial setup is complete.

6 Planning a gallery collection

A thoughtful structure helps visitors understand a collection and helps the owner maintain it over time. Start with the way people will look for photographs. Folder names and database details can follow that human organization.

6.1 Choose a structure people recognize

Common structures include:

By year and event A top-level year contains weddings, trips, celebrations, and projects. This suits family and historical archives.

By place Countries contain regions and cities. This suits travel, aviation, landscape, and documentary photography.

By client or project Each client contains assignments and delivery galleries. This suits professional work with separate access rules.

By subject Wildlife, architecture, portraits, aircraft, and similar subjects form enduring categories. Tags can provide a second way to browse across them.

By workflow Incoming, selected, delivered, and archive galleries represent stages. Visibility settings keep working collections private while finished selections become public.

Use galleries for strong navigational boundaries and tags for relationships that cross those boundaries. A photograph from Prague can live in *Travel / Czech Republic / Prague* and carry tags such as *architecture*, *night*, and *tram*.[7]

6.2 Decide how deep the hierarchy should be

Nested galleries support deep collections, yet most visitors browse more comfortably when each level has a clear purpose. Two to four levels serve many sites well. A deeper archive remains practical when breadcrumbs, meaningful titles, and consistent dates guide the reader.

Create a child gallery when it deserves its own title, description, cover, access policy, date range, or public address. Keep photographs together when a new level would add only one extra click.

6.3 Prepare titles and descriptions

A gallery title should make sense outside the Admin interface. A description can answer three questions:

- What does this collection show?
- Where and when was it created?
- What context makes the photographs meaningful?

Descriptions can be brief for casual albums and extensive for documentary work. Use the first sentence as the essential summary because visitors often scan before reading the full text.

[7] A photograph has one gallery location in the normal content model, while reusable tags provide several semantic connections. Copy and move tools support deliberate changes to physical organization.

6.4 Plan privacy before uploading

Choose whether a collection is public, unpublished for administrative preparation, or protected for a known audience. Parent-gallery policy can influence descendants, so place collections with similar audiences together where practical.

Examples include:

- a public portfolio with carefully selected photographs;
- a password-protected family branch;
- an unpublished client proofing gallery during preparation;
- a public event gallery with selected individual photographs marked private;
- an age-gated branch for content that needs visitor confirmation.

7 Creating and editing galleries

7.1 Create a gallery from Admin

Choose a parent gallery when the new collection belongs inside an existing subject, year, place, or client. Leave the parent empty for a top-level gallery. Enter a title, review the proposed folder or public path, and add the description, dates, and initial visibility.

After saving, the gallery becomes a real collection in the gallery tree. Its folder can receive photographs through the browser, filesystem transfer, discovery, or another supported upload workflow. The database stores its descriptive and presentation settings.

7.1.1 Creation sequence

1. Choose the intended parent.
2. Enter a visitor-friendly title.
3. Confirm the folder and public-path proposal.
4. Add a description and date range where useful.
5. Select visibility and access.
6. Save the gallery.
7. Add photographs or create child galleries.
8. Select a cover after suitable photographs exist.
9. Preview the result anonymously.

7.2 Edit identity and context

The editor lets you improve a gallery over time. Titles and descriptions can change as a collection develops. Date fields can represent a single day, a trip, a season, or a longer historical period. EXIF date suggestions can examine photographs and descendants, then offer an editable range for approval.

Use date suggestions as a helpful starting point. Scanned photographs, screenshots, edited exports, and cameras with an incorrect clock can produce dates that deserve human review.[8]

7.2.1 Review date suggestions across galleries

The dedicated **Gallery dates** maintenance page applies the same EXIF evidence across many galleries. It can review the whole installation or one selected gallery branch. Each row shows the current manual date range, the suggested range, the number of EXIF photographs and gallery nodes that contributed evidence, and editable **From**/**To** fields. Rows without an existing manual date are selected for application by default; rows that already have a manual date are left unchecked so a bulk review does not silently overwrite owner-entered chronology.

[8] Applying an EXIF-derived suggestion updates the proposed gallery fields through the existing editor workflow. The administrator remains responsible for the final dates shown to visitors.

Only checked rows are saved. Unchecked suggestions are ignored, and the administrator can edit either date before applying the selection. If older imported photographs do not have scanned EXIF capture dates yet, run the normal gallery image scan/import first. The same focused suggestion action is available inside an individual gallery editor and can refresh that editor in place.

Admin date fields keep the native browser date input as the submitted value but add a compact calendar opener plus **Today** and **Delete** helpers. **Today** writes the browser's current local calendar date; **Delete** clears the field. The enhancement is re-applied to forms loaded into the Admin side panel, while the underlying native date field remains usable without the extra JavaScript controls.

7.3 Gallery editor control reference

The gallery editor is the authoritative place for settings that belong to one physical gallery. The controls are grouped as Identity, Access, Display, Media, Images, Organizer, Renamer, and API-related tools where the corresponding optional feature is enabled. The following table summarizes the current gallery-owned controls in Version 0.97.

Area	Control	Current behavior
Identity	Title and description	Public editorial text. Maintained translations can be stored separately without changing the source text.
Identity	Manual date and date range	Optional event or shooting date plus supported start/end range. EXIF-derived suggestions remain editable before save.
Identity	Slug	Controls the public gallery URL identity. Keep established slugs stable when possible because bookmarks and external links can depend on them.
Identity	Folder name	Renames the physical gallery folder on disk through the normal mutation path. It is not only a display label.
Identity	Parent and sort order	Changes hierarchy and sibling ordering. Moving a gallery physically relocates its folder tree where the configured gallery root permits it.
Identity	Tags	Reusable tags separated by commas. Suggestions are weighted by nearby galleries, photographs, and folder context rather than being a flat random list.
Access	Visibility	**Public** is listed normally. **Unpublished** is hidden from normal listings but can still open from its normal URL when access rules allow it. **Private** is administrator-only except through supported direct-token access.

Area	Control	Current behavior
Access	Password lock	Independent from visibility. A public, unpublished, or private gallery can have its own password policy where the access schema is available. Leaving the new-password field empty preserves the existing password.
Access	Share link	Can be generated, regenerated, revoked, and optionally given an expiry. Stored token material is hash-protected; an older or non-decryptable token may need regeneration before a copyable link can be shown again.
Access	NSFW / 18+	Marks the gallery branch for anonymous age confirmation. The protection also applies to subgalleries and served media according to the effective access policy.
Display	Picture Game	Enables the comparison game for the gallery branch. Enabling the game also requires image voting to be available.
Display	Image voting	Enables public image likes for this gallery. Disabling it leaves stored scores intact but removes new vote controls and blocks new submissions.
Display	Show file names	Off by default. Raw uploaded filenames remain hidden unless enabled; custom photo titles and descriptions are still shown.
Display	EXIF/GPS display	**Use global default**, **Force on**, or **Force off**. The effective state controls public GPS pins, gallery EXIF maps, and GPS coordinate display.
Display	Gallery-card description format	Inherits Theme by default and can override the card layout for this gallery where supported.
Display	Contained-picture badge	Inherits Theme or overrides the stacked-picture icon and contained-image count shown on gallery cards.
Display	Lightbox browsing mode	Inherit Theme, classic single image, picture strip, or 3D carousel. Fullscreen and slideshow behavior remain available across the modes.
Display	Custom grid	Optional columns and rows for this gallery. A gallery without an explicit grid inherits the nearest parent grid that permits subgallery inheritance, then falls back to Theme.

Area	Control	Current behavior
Display	Responsive thumbnail bounds	Optional minimum/maximum thumbnail-size guardrails. A save can copy gallery bounds recursively to descendants; recursive save is off by default.
Media	Title picture	Automatic or a selected photograph, including eligible photographs from descendants when the picker offers them.
Media	Uploaded gallery thumbnail	Separate gallery-card media asset stored independently from ordinary gallery photographs.
Media	Background source	Inherit Theme, upload a new background, choose an existing gallery image, or generate a collage from public galleries.
Media	Branding assets	Optional gallery-specific banner/logo/separator assets override Theme fallbacks only for that gallery where configured.
Media	Flight route	Manual route text supports local identifier lookup and explicit `NAME@latitude,longitude` points. SimBrief can save an OFP, description draft, and resolved route coordinates.
Images	Scan/import	Re-scans the physical gallery folder and synchronizes supported media into the catalog.
Tools	Organizer, Renamer, API	Date organizer, physical Media Renamer, upload API key management, AI regeneration, and gallery migration appear only when their schemas/features are available.

Important. Gallery-specific controls are intentionally not flattened into the global Settings hub. Inheritance is part of the data model, so a global search result may link to the gallery editor without turning that gallery-owned value into a site-wide setting.

7.4 Choose a cover and visual identity

A cover acts as the gallery's visual invitation. Choose an image that remains recognizable as a thumbnail and represents the collection fairly. Faces, strong silhouettes, clear subjects, and uncluttered compositions often work well at small sizes.

Gallery branding can provide a logo, banner, background, separator, or presentation override according to available controls. Begin with inherited theme styling, then add gallery-specific assets where a collection has a genuine identity, such as a wedding, exhibition, client, or long-running project.

7.5 Move and reorder galleries

Moving a gallery changes its place in the hierarchy and can affect inherited access or presentation. Review the destination before confirming the move. Reordering changes the sequence shown among siblings and supports a curated reading order.

Ordering may be chronological, geographical, narrative, alphabetical, or manually featured. Within one parent gallery, keep the choice consistent enough that visitors can anticipate the next item.

A signed-in administrator can also reorder the direct subgallery cards on the public gallery page. The compact public reorder toolbar is limited to the cards visible on the current pagination page. The save request includes that page's offset and visible count, and the server refuses the change if the submitted IDs no longer match the same current slice. It changes sibling `sort_order` values only; it does not change gallery parents or nest galleries.

When an administrator is signed in while viewing a public gallery, direct child galleries that have a filled **From** date can also be preview-sorted from oldest to newest or newest to oldest. Child galleries without a usable date keep their existing positions. The preview is temporary until **Save** is chosen; saving writes the result as the real sibling order seen by all visitors. At least two dated direct child galleries are required before the date order can be saved.

7.6 Delete a gallery thoughtfully

Gallery deletion can affect source files, descendants, metadata, thumbnails, links, and visitor bookmarks. Review the exact selected gallery and its children, create a backup, and use the normal Admin deletion workflow. A temporary visibility change can provide a safer preparation period when a gallery may still be needed.

The gallery trash bin is enabled by default. Deleting a gallery through a normal administrator gallery-delete action moves the selected folder and its complete descendant tree out of the live gallery root, records the metadata required for recovery, and lists the entry under **Maintenance > Trash**. Public pages stop discovering that subtree immediately, but an administrator can restore it to its original path while that path remains free. Restore never overwrites a new live gallery occupying the same location.

From the Trash panel, choose **Restore**, **Delete permanently**, or **Empty Trash**. Permanent deletion and Empty Trash cannot be undone. Empty Trash works in bounded batches so large trash collections do not require one unbounded request. Individual-photo deletion is not part of this gallery-level recovery feature and remains permanent. A backup remains necessary protection against storage failure, administrative permanent purge, or damage outside PHP Gallery.

8 Smart Galleries

A Smart Gallery is a saved query, not a folder. Open **Admin > Galleries > Smart Galleries**, add conditions, and combine them with nested **AND**, **OR**, and **NOT** groups. Fields cover source galleries and descendants, tags, capture dates, EXIF/GPS, text, AI metadata, duplicate state, file characteristics, and private editorial ratings.

Use **Preview** to see the current match count and real matching photo cards, then save and optionally publish. Images and files remain in their physical galleries, so metadata or rating changes alter membership immediately. The editorial rating is private and independent from visitor voting. Administrators may also use **Save search as Smart Gallery** from public search.

Choose **Unlisted** for URL-only access, **Root gallery** to show the Smart Gallery on the homepage, or **Subgallery placement** to let physical galleries attach it. In each physical gallery editor, select whether an attachment belongs **Above gallery content** or **Below gallery content** and give it an order value. Above attachments render first, then ordinary subgalleries and photos, then Below attachments. Equal order values are resolved consistently by Smart Gallery ID. Existing attachments default to Below with order 0.

The same virtual gallery may be attached beneath several physical galleries, with independent placement and order in each parent. The Smart Gallery editor's **Used in physical galleries** section shows every parent and lets you save its placement/order or **Detach** it without changing other parents. When the editor is opened in the Admin side panel, these actions stay in that panel; ordinary form submission remains available when JavaScript is disabled. A public Smart Gallery placed beneath a private or otherwise restricted parent still inherits that parent's public access boundary for the card.

Public results and counts always apply the same source-access policy as ordinary public gallery content. A matching photograph contributes only when its physical gallery is publicly listed and accessible to the current visitor and the photograph itself is public. Existing password, share-link, inherited visibility, unpublished-gallery, and age-restriction rules therefore continue to apply. Query-string routes work without rewriting and clean `/smart/<slug>` URLs are optional.

8.1 Relationship safety and repair

A Smart Gallery must not resolve back into a physical gallery that attaches it. The server checks the complete relationship graph whenever rules, attachments, or gallery hierarchy are changed; browser filtering is not treated as security. This includes indirect paths through physical descendants and other attached Smart Galleries. Filesystem-derived hierarchy synchronization and public-path parent repair validate their complete proposed parent map before any new parent link is written. The current rule editor can reference physical galleries but does not offer a direct Smart-Gallery-reference condition.

A complete Admin drag-and-drop tree is validated once in its final form. The subsequent folder moves use that prevalidated map and defer hierarchy synchronization until the final tree exists, so a temporary intermediate state cannot reject an otherwise valid batch. Every committed physical hierarchy mutation clears the request-local Smart Gallery graph cache before later graph-dependent work continues.

If older or manually modified database data already contains a cycle, public rendering stops that invalid relationship instead of recursing. The Admin attachment list marks the relationship for repair, and **Detach** remains available. Validation and runtime traversal both use conservative depth/node limits and request-local deduplication, so malformed data cannot repeatedly expand the same Smart Gallery. The normal result query remains paginated and does not copy all

matching photographs into PHP merely to perform cycle detection.

Migration `202608170002_smart_gallery_attachment_ordering.php` adds the per-parent placement and order fields. Until that migration is available, existing attachments keep their historical Below behavior, while Admin placement mutations are refused rather than partially saved.

8.2 Presentation settings

By default a Smart Gallery follows the current Theme and site presentation. Enable **Override Theme defaults for this Smart Gallery** when this virtual collection needs its own columns, rows, pagination, thumbnail size limits, thumbnail renderer, gallery-card layout, metadata overlays, lightbox behavior, slideshow, download, or voting behavior. Sorting and direction remain part of the Smart Gallery definition.

A Smart Gallery does not inherit presentation from the physical gallery in which its card appears. One Smart Gallery can appear beneath several physical galleries, so there is no single parent presentation with clear precedence. When no override exists, or when stored presentation data is missing or invalid, the current Theme/site defaults remain the fallback. Physical gallery and photo thumbnail limits still win if a Smart Gallery limit conflicts with them.

8.3 Lightbox across result pages

The large viewer is not limited to the photographs rendered on the current HTML page. Smart Gallery cards remember each photograph's position in the complete authorized result order. As the visitor moves forward or backward, the browser requests nearby metadata in small authorized groups from the server. The server repeats the same Smart Gallery rules, source access checks, sorting, and stable tie-break used by the page.

This keeps large virtual collections practical: opening a Smart Gallery does not preload all originals or all result metadata. Existing keyboard, touch, fullscreen, zoom, close/reopen, and slideshow behavior is shared with ordinary galleries. If slideshow is disabled for that Smart Gallery, its slideshow controls are omitted while normal galleries keep their usual defaults.

8.4 Downloads and safety limits

When site downloads and the Smart Gallery download option are both enabled, the server can prepare a ZIP from the current authorized matching originals. It re-evaluates the saved rules and access policy instead of exposing filesystem paths to the browser. The archive path refuses collections above 5,000 matching images and also enforces an aggregate original-file byte ceiling. Existing installations use a 2 GiB ceiling unless `smart_gallery_zip_max_source_bytes` is set in `config.php`. Concurrent requests for the same archive signature are serialized with a file lock; after the lock is obtained the cache is checked again, a unique partial ZIP is written, and the completed archive is published with an atomic rename. Failure logs keep only bounded reason/class diagnostics rather than raw exception messages or private server paths.

Rules are readable versioned JSON, never user SQL: fields/operators are allowlisted, values are prepared parameters, depth is limited to five, and no more than 50 conditions are accepted. Deleted references match nothing and malformed or unknown rule versions are rejected. The normal migration workflow adds Smart Gallery presentation storage; existing Smart Galleries inherit Theme/site defaults until an override is saved.

9 Adding, describing, and organizing photographs

9.1 Choose an upload method

Browser upload suits everyday additions. Select a gallery, choose files, review the batch, and follow progress while the server validates media and prepares records and thumbnails. Filesystem discovery suits large existing folder trees or transfers made through FTP and hosting tools. Automation and WebDAV suit repeated or mobile workflows after their scoped credentials are configured.

For a large event, upload in meaningful groups. Smaller groups make errors easier to identify and allow titles, ordering, and visibility to be reviewed incrementally.

9.2 Write useful photo titles

A title identifies the photograph quickly. A description or caption can explain people, place, activity, technical context, or a story outside the frame. Useful examples include:

- *Charles Bridge before sunrise*;
- *Arrival of the first train at the restored station*;
- *Family group in the garden, June 1998*;
- *Final approach during the Saturday display*.

Meaningful text improves search, accessibility, later identification, and the experience of readers who want more than a visual grid. File names can remain operational details when public display settings favor curated titles.

9.3 Use tags as a second organization layer

Create a small vocabulary before adding many near-duplicate tags. Prefer one stable term, such as *aircraft*, over several accidental variants. Tags can represent subjects, people, places, techniques, equipment, seasons, or editorial status.

For a travel archive, galleries can represent trips and tags can represent recurring themes. For a family archive, galleries can represent years and events while tags identify people. For a professional portfolio, galleries can represent clients and tags can identify deliverable type, location, and specialty.

Tag entry fields can suggest existing tags while you type. Reusing a suggestion avoids accidental spelling variants and keeps tag landing pages coherent. The suggestion is an editing aid only; the ordinary save action still decides which tags are stored for the gallery or photograph.

9.4 Arrange the viewing order

Ordering turns a collection into a sequence. Start with an inviting image, establish place or context, vary wide and close views, keep closely related details together, and finish with a natural conclusion. Chronological order works well for events, while visual rhythm can suit portfolios and exhibitions.

The public reorder controls help administrators adjust visible cards. Photo reordering on a public page is also scoped to the current visible pagination slice and saves through the authenticated

image-order endpoint. When Picture Manager is active, dragging selected photo cards over a visible child-gallery card changes the gesture from reordering to a validated move into that child gallery. Pagination can divide a large collection, so review transitions around page boundaries as well as the first page.

9.5 Review EXIF and location information

EXIF can enrich a photograph with capture date, camera, lens, exposure, and GPS coordinates. This information supports maps, date suggestions, duplicate evidence, and technical context. Review GPS visibility before publishing photographs made at homes, schools, private workplaces, or sensitive natural locations.

10 Publishing, sharing, and audience scenarios

10.1 Public portfolio

A public portfolio benefits from a small number of strong top-level galleries, consistent covers, concise descriptions, and careful ordering. Use tags for specialties that cross projects. Keep source archives in private or unpublished branches and copy selected work into public presentation galleries where the workflow calls for separate curation.

Test the portfolio on a phone, a high-density display, and a slower connection. Responsive thumbnail selection provides the default balance. Progressive sharpening can make small thumbnails populate first for installations that prioritize early visual response.

10.2 Private family archive

Organize by year, family branch, or event. Use password-protected parent galleries to establish an understandable private area, then place related descendants within it. Add names and dates while memories are available. Scanned photographs benefit especially from human descriptions because embedded camera metadata may be absent.

Share the protected address and password through an appropriate private channel. Explain that recipients can navigate child galleries without receiving an administrator account. Periodically review whether old links and passwords still serve the intended audience.

10.3 Client delivery

Create one protected gallery per client or assignment. Upload proofs, organize them into useful groups, and use descriptions to explain selection or download expectations. Voting can support lightweight preference gathering when enabled for that gallery. A final delivery gallery can contain approved files in the intended order.

Share links offer another controlled entry method where configured. Test the exact client journey in a private browser window before sending the address.

10.4 Event and community gallery

An event gallery can begin unpublished while photographs are arriving. Use child galleries for days, stages, teams, locations, or activities. Publish the parent when the first useful selection is ready, then add later groups without changing the familiar public address.

Public voting can highlight audience favorites. Tags connect recurring participants or subjects. Downloads provide a convenient collection transfer when the event policy allows it.

10.5 Travel and location archive

Country, region, city, and trip galleries provide natural navigation. Date ranges establish chronology, while GPS maps create a geographical reading path. Review coordinates for accommodation and private locations. A descriptive introduction can explain the route, purpose, and historical context of a trip.

10.6 Historical and institutional archive

Historical collections benefit from careful source notes, uncertain-date wording, names, locations, and provenance. Use descriptions to distinguish confirmed facts from family or institutional recollection. Tags can connect people, buildings, organizations, and periods across many galleries.

Create backups before large metadata projects and export information through available tools when planning external preservation. The gallery can provide an accessible presentation layer while original preservation practices continue alongside it.

11 Understanding your gallery data

Gallery owners interact with two cooperating stores: ordinary files and a database. Understanding their roles makes backup, migration, and troubleshooting easier.

11.1 What lives in gallery folders

The gallery folders contain the photographs and folder hierarchy, so the core collection remains recognizable through FTP, a hosting file manager, a backup tool, or a local copy. Moving or renaming files outside the application can require discovery or synchronization so the database learns the new state.

11.2 What lives in the database

The database remembers titles, descriptions, dates, visibility, passwords and access configuration, public paths, ordering, tags, votes, image dimensions, EXIF indexes, checksums, thumbnail records, administrator accounts, settings, audit information, and workflow state.[9]

The database makes search and administration fast. It also allows a file to have a friendly title, rich description, tags, a chosen order, and access rules without changing the original image bytes.

11.3 How users benefit from the database

Everyday benefits include:

- searching words across gallery and photo descriptions;
- opening a gallery through a stable public path;
- displaying photographs in a curated order;
- remembering tags and visitor votes;
- protecting a gallery with inherited access rules;
- finding exact duplicate content through stored checksums;
- preparing suitable thumbnails without re-reading every source file on every page;
- recording migrations and maintenance decisions with an audit trail.

11.4 Backup as one complete set

A complete backup includes the gallery tree, database export, local configuration, and installation-owned custom assets. Give these parts the same backup date so they describe one coherent state. Store a copy away from the web server and test restoration in a separate location.

[9] Password values use hashes or protected configuration appropriate to their purpose. A database export still contains sensitive operational information and deserves secure storage.

12 Everyday ownership and care

12.1 After every important upload

Review the destination gallery, photo count, ordering, titles, visibility, and thumbnail appearance. Open several photographs in the lightbox and check the gallery anonymously. For a protected collection, repeat the password or share-link journey from a fresh private session.

12.2 Monthly review

A monthly maintenance check should include:

1. Review System Health and pending migrations.
2. Check recent Admin audit events.
3. Inspect available application updates and their release information.
4. Confirm scheduled or manual backups completed.
5. Open representative public and protected galleries.
6. Review thumbnail maintenance when source files or rendering settings changed substantially.
7. Remove or revoke credentials and share methods that have completed their purpose.

12.3 Before a major public launch

Check titles, descriptions, cover images, spelling, mobile layout, privacy, GPS exposure, downloads, maps, voting, search results, and contact expectations. Test through a slow network profile if the audience may use mobile connections. Create a backup immediately before the launch state.

12.4 When another person helps administer

Give each administrator an individual account where the account model supports the intended workflow. Explain gallery scope, privacy expectations, naming conventions, tag vocabulary, backup responsibilities, and deletion procedures. Individual accounts improve accountability in audit records and keep personal duplicate-review ledgers separate.

13 Public visitor experience

13.1 What a visitor sees

A visitor opens the site at the home page or follows a direct link to a gallery or photograph. The page presents the site's name, navigation, gallery title, description, breadcrumbs, tags, child galleries, and photographs that the visitor is allowed to see. Protected content asks for the appropriate access step before revealing private material.

Each gallery has a public address that can be bookmarked or shared. A direct photo address opens the gallery context and leads the visitor to that photograph. Meaningful titles and stable public paths help links remain understandable in messages, bookmarks, and search results.

13.1.1 Typical visit

One common visitor path is:

1. Open the home page and choose an interesting gallery cover.
2. Read the gallery introduction and follow child galleries or tags.
3. Scroll through smaller preview images.
4. Open one photograph in the large viewer.
5. Move through nearby photographs with buttons, keys, a strip, or the selected viewing mode.
6. Open a map, vote, search, or download when the gallery offers those actions.
7. Use breadcrumbs to return to a broader collection.

13.2 Gallery hierarchy and pagination

13.2.1 Direct content and stable ordering

A gallery can show smaller galleries first and photographs afterward. For example, *Italy 2026* can contain *Rome*, *Florence*, and *Venice*, while also showing a few overview photographs directly on the Italy page.

Pagination divides a long collection into manageable pages. This helps the first page appear promptly and keeps scrolling comfortable. A small exhibition may look best on one page. A wedding with 1,500 photographs will usually benefit from pages or child galleries. The large photo viewer can continue through the wider gallery sequence while loading additional information as needed.[10]

Long public listings include a floating **Top** control after the visitor has scrolled into the gallery listing. It uses smooth browser scrolling, disappears outside the active listing, and stays hidden while the lightbox or fullscreen viewer is open so it does not compete with photo controls.

13.3 Search, tags, and navigation

Search can look across the whole public site or stay within the current gallery and its children. A visitor looking at *Family archive* can search only that branch for a person's name, while a visitor

[10] Pagination changes how many cards appear at once. It leaves the underlying photographs and their order intact.

on the home page can search across every public collection. The current search service begins at two visible query characters, returns a bounded mixture of gallery and photo results, and ranks stronger title/name/tag matches ahead of weaker text matches. The default request returns up to 12 merged results and the service clamps a request to at most 30 results.

Searchable evidence includes public gallery/photo titles and descriptions, image filenames, gallery and photo tags and their public descriptions, active maintained-language content where its schema is available, and internal AI searchable text when AI metadata is ready. Access filtering happens as part of the query: private, inaccessible, or non-public image content is not returned merely because its text matches. A branch search also constrains the listing context to that gallery subtree rather than searching the whole catalog and hiding unrelated matches afterward.

Tags provide another route through the archive. A *night photography* tag can connect Prague, London, and Tokyo even when those photographs live in separate travel galleries. Breadcrumbs show the current place in the hierarchy, and favorite shortcuts place important galleries directly in the site header.

14 Media and gallery administration

14.1 Admin workspace

After signing in, an administrator sees private controls for the dashboard, galleries, photographs, theme, tags, maintenance, updates, logs, and account settings. Many edits open in the right-side panel, leaving the gallery visible while one item is changed and saved.[11]

The public gallery itself also exposes contextual Admin controls while a normal administrator session is active. Gallery and photo pencil controls open the same full editors in the side panel. Compact gallery removal uses the recoverable trash workflow when Trash is enabled; disabling Trash returns future gallery removals to the legacy permanent subtree-deletion service. Compact photo removal is deliberately labeled **Remove photo ... from CMS**: that quick public-page action removes the catalog image row and does not run the full source-file deletion service. Use the normal photo/bulk deletion workflow when the original file and generated derivatives should also be deleted. Anonymous-preview mode hides these contextual Admin controls.

The Admin gallery table can filter rows by visibility: all, unpublished, public, or private. Filtering is a browser-side view aid, not an access-policy change. A row hidden by the filter is also unchecked, and the master selection is cleared, so a hidden stale checkbox cannot accidentally enter the next bulk action. The summary reports the displayed/eligible gallery count.

The gallery tree supports per-branch expand/collapse plus **Collapse all** and **Expand all**. It remembers which gallery branches the administrator collapsed. The browser sends the collapsed gallery IDs through an authenticated CSRF-protected action and the application stores that UI state, so returning to the Admin tree keeps the same branches closed rather than expanding the entire hierarchy again. Descendants hidden by a collapsed branch are also unchecked before bulk work. Reordering refreshes the tree controls when a gallery gains or loses children.

14.2 Reliable in-place side-panel changes

When JavaScript is available, a persistent action started in the Admin right-side panel completes in that panel. The drawer stays mounted and open, the address in the browser remains unchanged, and the page is not reloaded merely to show the result. This applies to gallery create/edit/move/delete, image upload/edit/delete/move/reorder/visibility/NSFW/cover actions, scan/import, EXIF date suggestions, thumbnail creation, Media Renamer, Metadata Organizer, Duplicate Photo Detector review/deletion actions, Upload Automation credentials, Gallery Migration target pulls, tag edits, and Smart Gallery create/edit/placement/detach/duplicate/delete actions. A standalone Admin page and ordinary POST/redirect behavior remain available when JavaScript is disabled or the route is opened directly.

The saved database/filesystem state remains authoritative. After persistence, the server describes the changed entity, the panel fragment that may need re-rendering, and every affected public context. Stable gallery, image, tag, and Smart Gallery identifiers are used instead of assuming that the old URL still names the saved object. This matters after a gallery folder/slug rename, gallery reparenting, or tag slug change: the application can fetch the new canonical render source while leaving the visible browser address untouched. Source and destination galleries are both described for moves; old/new parents and the moved gallery are described for reparenting; Smart Gallery placement changes describe every affected root or physical parent context.

A successful fragment request is not by itself proof that the change is visible. The refreshed server-

[11] A full Admin page remains available for workflows that need more space or when browser enhancement is unavailable.

rendered HTML must satisfy an operation-specific observable condition where one exists. Examples include gallery/card presence or absence and full counts, exact effective visibility, saved gallery revision, image absence/visibility/NSFW state, gallery-wide image revision for off-page changes, cover image identity, persisted image order, tag identity, and Smart Gallery placement/order. Pagination-aware checks accept a directly visible matching card before using aggregate counts, while an off-page result still waits for the authoritative total to converge. Thumbnail cache repair is the intentional exception: it refreshes affected contexts but has no stable database-backed public-HTML condition.

Independent render requests can briefly return stale data on shared hosting. PHP Gallery therefore performs one centralized bounded retry sequence only after the mutation itself succeeded and the declared condition is not yet observable. Each completion receives a new operation generation; a newer mutation aborts or supersedes older fetches, retry wakeups, and panel responses for the same context. Structurally valid but stale HTML is verified before replacement, so an older response cannot overwrite a newer visible state. Refreshes bypass browser caches and preserve applicable pagination, sorting, language, filter, active-tab, drawer-scroll, and page-scroll state.

If persistence succeeds but the refreshed view cannot be verified within the bounded retry budget, the panel reports a synchronization problem and keeps the saved server change. It does not misreport the mutation as failed, silently reload the page, navigate away, or rewrite browser history. Authentication, authorization, CSRF, schema readiness, path safety, and validation failures return an expected structured error to the enhanced workflow and are not retried. Dynamically refreshed controls remain active because the panel uses lifecycle-safe delegated handling rather than bindings that disappear after fragment replacement.

Browser-assisted create/upload has additional safeguards. Selecting browser processing is an explicit execution choice: a missing browser capability or failed worker preparation stops before persistence instead of silently switching to server thumbnail generation. PHP validates that every required prepared thumbnail size/format is present before it stores originals. Once browser persistence starts, direct-page fallback metadata is never interpreted as permission to replay the classic create/upload request, preventing duplicate galleries and duplicate photographs. The configured ZIP batch size is a soft packing target: one atomic photograph package that exceeds that target can travel alone in one batch, but the effective PHP upload ceiling remains a hard limit. Windows upload automation may send the complete JPEG+WebP compatibility matrix; a modern WebP-only server accepts required WebP variants and safely skips valid extra JPEG variants, while malformed, mismatched, unknown-size, or unsupported thumbnail payloads remain hard failures.

14.3 Central Settings hub

The Admin navigation includes a central **Settings** page for site-wide configuration. It is divided into General, Public appearance, Content, Media and browsing, Uploads and automation, Privacy and diagnostics, and Advanced. Each section has its own stable address, so refresh, bookmarks, browser history, and the no-JavaScript fallback return to the same category.

A Spotlight-style field searches settings as you type. It matches names, descriptions, categories, and technical names, including controls that are owned by specialist Admin pages. Selecting a result opens the relevant section or specialist page and focuses the matching control. Arrow keys, Enter, Escape, and the clear button can be used without leaving the keyboard.

Frequently used global settings can be edited directly from the hub, including the site name, public language, URL rewriting, public search where available, thumbnail rendering, the global EXIF/GPS display default, and development diagnostics. Other settings remain on their specialist pages when they need more context, credentials, file uploads, destructive actions, or larger editors. Examples include Theme layout, uploads, telemetry, account security, Google/OpenAI

configuration, custom CSS, branding, language packs, and database maintenance.

The hub does not flatten existing inheritance. Tag-page layout can continue inheriting the global grid and card design, while gallery-specific card, lightbox, and EXIF/GPS choices stay with the gallery that owns them. Context-specific gallery and photo values are not turned into global settings.

Secrets are never shown in search results or summaries. Passwords, tokens, API credentials, and similar values appear only as safe states such as Configured, Not configured, or a link to their specialist page.

14.4 Tag metadata administration

Admin > Edit tags is the central editor for the reusable tag vocabulary. The list can be sorted by total usage or alphabetically and shows each tag's current gallery-plus-photo use count. Selecting a tag exposes its canonical lowercase name, clean public URL slug, and optional public description for the tag landing page. The same view lists the galleries and photographs that currently use the tag, with links back to the relevant public or photo-editor context.

Renaming a tag or changing its slug updates the shared tag record, so every gallery/photo assignment continues to reference that one reusable tag. Names and slugs are normalized to the safe tag convention on save and a duplicate slug is rejected. Deleting a tag is different: the application transactionally detaches it from both galleries and photographs and then removes the tag record. Use **View public tag** to review the landing page and **Configure tag display** to open the Theme controls for tag-page/card and hero-tag presentation.

14.5 Feature switches

Appearance > Features provides global switches for optional parts of the application. Registered features are enabled by default so an upgrade does not silently remove behavior that an existing installation already uses. An administrator can disable individual visitor features, maps and flight-simulation tools, AI/upload integrations, or special-purpose Admin tools when a simpler installation is preferred.

A disabled feature is hidden from normal navigation and its guarded routes refuse the feature workflow. Disabling a switch does not delete the feature's code, photographs, database records, credentials, or configuration. Re-enabling it therefore restores the normal entry points subject to the same schema, access, and configuration checks as before. Examples of switchable features include public search, lightbox browsing modes, Picture Manager, voting, Picture Game, downloads, gallery and flight maps, navigation data, SimBrief, OpenAI assistance, local AI metadata, API uploads, mobile WebDAV, gallery migration, Media Renamer, and anonymous telemetry.

14.6 Gallery management

Gallery administration covers the full life of a collection: creation, discovery, naming, description, dates, tags, privacy, passwords, branding, covers, layout, ordering, moving, and eventual deletion. A child gallery can inherit useful choices from its parent. For example, every gallery inside a private family branch can follow the parent's protection, and an individual child can use its own presentation choice where the editor offers an override. The bulk gallery screen provides corresponding multi-gallery operations where the same change is appropriate for several collections at once.

Gallery descriptions can present safe external links written as ordinary Markdown links, `[link=URL]label[/link]`, `[url=URL]label[/url]`, or the short forms whose visible text is also the target. Only HTTP and HTTPS targets are linked; other schemes remain inert text. Recognized services use bundled local brand symbols. For other public hosts, saving the gallery can retrieve a small validated favicon into the local gallery cache. Retrieval is hostname-deduplicated, bounded by size, time, redirect, and retry limits, rejects private or reserved network destinations, and never occurs while a visitor renders the public description. If the cache or its database table is unavailable, the link remains usable without an icon.

When clean public-path storage is available, **Regenerate clean public paths** rebuilds the stored gallery and image paths from the current hierarchy and reports separate gallery/image counts. Reordering and selected editor saves also refresh paths when their changes require it. This maintenance action is useful after hierarchy repair, migration, or older imports; it does not turn URL rewriting on by itself, so query-string routing remains the fallback when rewriting is disabled or unsupported by the host.

14.7 Metadata organizer

The gallery editor includes an **Organizer** workflow for turning a flat set of direct photographs into date-based child galleries. The current implementation groups by EXIF capture date already stored in the image database. Location-based secondary grouping is reserved for future use and is not an active organizer strategy in Version 0.97.

Choose minimum and maximum EXIF dates, then build a preview. Previewing reads database metadata in small batches and does not scan, rename, or move source files. The draft shows the number of direct photographs, matching candidates, proposed child galleries, and photographs ignored because they have no usable EXIF date or fall outside the chosen range. A minimum date is particularly useful for excluding photographs from a camera whose clock was unset.

For each accepted calendar date, the proposed child folder name is the stable ISO date `YYYY-MM-DD`. Its display title uses the gallery date-display formatter and falls back to a human-readable day/month/year value if that formatter cannot supply a label. Before proposing a new child, the organizer looks for an existing direct child with the same organizer title and reuses it. This makes a repeated preview/apply cycle append to the same daily destination instead of creating another equivalent day gallery.

Applying the draft requires a separate confirmation. PHP Gallery creates or reuses the proposed date child galleries and moves matching photographs through the same physical move service used by the ordinary bulk image mover. Originals and known generated derivatives therefore move together with the database ownership update. Large applies are split into bounded browser-driven requests instead of one long PHP request. Review the draft and maintain a backup before reorganizing a large gallery.

14.8 Media renamer

Galleries > Media renamer, and the renamer panel available for an individual gallery, can normalize physical image filenames from gallery context and the current photo order. This is a filesystem operation, not a cosmetic title change, so the tool always begins with a dry-run plan. The preview lists each old and proposed filename and marks files that already match, are missing, would collide, or must be skipped by a safety rule.

Only direct image files inside the selected gallery folder are eligible. Missing sources, paths outside the gallery, and unsafe targets are not renamed. If the preferred target name is already

occupied but a safe deterministic suffix can resolve the collision, the preview shows that adjustment. Applying a plan requires confirming that the preview was reviewed.

The default pattern is `\{gallery_path\}_\{seq4\}`. A custom pattern may use `\{gallery_path\}`, `\{gallery_title\}`, `\{photo_title\}`, `\{old_name\}`, `\{old_stem\}`, `\{image_id\}`, and sequence placeholders `\{seq\}` through `\{seq5\}`. The numbered sequence variants pad to two, three, four, or five digits. Textual components are normalized to lowercase filesystem-safe ASCII-style underscore stems, the source extension is preserved in sanitized form, and an empty custom pattern falls back to the default.

After a successful physical rename, PHP Gallery keeps the database row and public path references aligned, updates derived titles where the renamer owns them, moves or invalidates generated derivatives as appropriate, and removes stale cached ZIP archive records that refer to the old source identity. Derivative cleanup warnings do not justify reverting an otherwise completed source rename because reproducible derivatives can be generated again. For site-wide selections, the browser advances the work in bounded portions and shows the result for each gallery.

14.9 Image management

Photographs can be uploaded in the browser, copied into folders and discovered, received through configured automation, transferred from another compatible installation, or added through a scoped mobile workflow. After a photograph arrives, the gallery records its size and available camera information and prepares the smaller display copies used throughout the site.

The individual photo editor currently covers title, description, maintained-language variants, visibility, sort order, tags, NSFW / 18+ state, the private editorial rating used by Smart Gallery rules, optional per-photo responsive-thumbnail bounds, and read-only EXIF/GPS details. Where local AI metadata exists, the editor shows the generated model/source, searchable internal text, and raw metadata JSON separately from the public caption. OpenAI controls insert reviewable drafts into editable text fields and do not replace saved editorial text automatically.

Bulk image operations are intentionally narrower than the individual editor and perform server-side ownership checks again. Current bulk actions include:

- move selected photographs to an existing gallery;
- create a new destination gallery and move the selection into it, with cleanup of an empty newly created destination if the move fails before files are committed;
- delete selected photograph records together with their owned originals and known generated derivatives through the normal deletion service;
- set a selected photograph as the gallery title/cover image;
- change photo visibility among the supported draft/public/private states;
- turn per-photo NSFW protection on or off when its schema is known;
- create thumbnails for the selected photographs.

Photo ordering can be changed by dragging the row handle, and the gallery can be sorted by filename from the Name column. Changes are saved through the normal ordering route rather than being a browser-only arrangement.

When the optional **Picture Manager** feature is enabled, a normal signed-in administrator can select photographs directly in the public gallery grid. Checkmarks select individual cards; Shift-

click selects a range; Ctrl-click or Cmd-click toggles a card; **Select all**, **Clear**, and Ctrl/Cmd+A support larger visible-page selections. Dragging an unselected photo starts with that photo as the selection, while dragging an existing selection moves the selected photographs together. A visible child-gallery card becomes a drop target, and dropping there performs the same validated move as the destination picker.

From the toolbar the administrator can move the selection, copy it to an existing gallery, create a child gallery by copying the selected originals and metadata, or share the selection. **Share** fetches the authorized browser-displayable image versions as browser `File` objects and opens the native Web Share sheet when the browser/device supports multi-file sharing. The browser chooses the available receiving apps, so PHP Gallery cannot force Instagram Story, Reel, or another specific target. If native sharing is unavailable, cannot accept the files, or fails after preparation, the manager starts the authenticated selected-photo ZIP download as a fallback. The same ZIP endpoint can therefore serve both an explicit selected-photo download and share fallback. Ordinary visitors never receive these controls, and the server revalidates the logged-in account, CSRF token, source ownership, and destination gallery for every mutation or ZIP request.

14.10 Uploads and discovery

Filesystem discovery is useful when photographs have already been copied to the server through FTP, a hosting file manager, or a restored backup. The discovery screen shows what the application found and lets the administrator bring those folders and files into the catalog. Discovery runs as a session-backed browser job in small AJAX batches rather than scanning the entire tree during the initial Admin page request. The current default examines up to 80 directories per batch, accepts a bounded batch size up to 300, and expires abandoned discovery job state after two hours. This keeps a large folder tree responsive on slower hosting while still showing progress and a final actionable result list.

The scanner walks below the configured gallery root and does not follow directory symlinks. Hidden directories and known generated/cache directory names such as `cache`, `thumbs`, `thumbnail`, `thumbnails`, `preview`, and `previews` are ignored. A new candidate branch must contain at least one supported image somewhere below it. Empty structural parent folders can still become galleries when they are needed to preserve the hierarchy above image-bearing descendants. A folder containing only a `gallery.json` sidecar and no supported image anywhere in its branch is reported as metadata-only rather than imported as a gallery by the current discovery workflow. Already indexed gallery folders are suppressed from the candidate list, and an import candidate whose display title would duplicate an existing sibling title is not silently created.

For each unmanaged result, the administrator can import the folder tree in place, move supported photographs from selected unmanaged folders into an existing gallery, or delete selected unmanaged folders from disk. Import expands the selection in parent-to-child order and can include missing image-bearing ancestors and descendants. The move action avoids overwriting an existing destination filename by choosing a deterministic unique name, scans the destination gallery after successful moves, and removes source directories only when they become empty. The discovery-specific disk-delete action is deliberately restricted to safe non-root unmanaged paths below the gallery root and refuses folders that are already represented by database galleries. Use the normal gallery deletion workflow for an imported CMS gallery.

14.10.1 The gallery.json metadata sidecar

PHP Gallery keeps its active catalog and relationships in the database, while an optional `gallery.json` file beside a gallery folder preserves useful editable metadata with the filesys-

tem. Normal gallery saves write the sidecar back to the gallery folder. Current sidecar output includes the title, description, normalized gallery tags, visibility, order, voting and filename-display state, maintained content language/translations when available, manual gallery dates, selected card/count/lightbox/grid overrides, responsive-thumbnail bounds, access/listing policy, cover references, and configured gallery branding assets where those schemas are available. Sidecar writing follows the same schema checks as the corresponding gallery features.

During discovery and missing-parent repair, PHP Gallery can read supported values from the sidecar instead of deriving everything from the folder name. Imported rows conservatively default to unpublished when no visibility is specified. The current import path restores the sidecar fields explicitly supported by the importer, including title/description/localization, visibility, manual dates, voting/filename presentation, selected card/count/lightbox/grid/thumbnail overrides, access/listing policy, order, and branding paths. The sidecar should therefore be treated as portable gallery-folder metadata, not as a complete database backup: relational state, credentials, logs, votes, game state, Smart Gallery definitions, and other database-owned records still require the normal backup procedure.

> **Security note.** The **Delete from disk** action on the discovery screen is a real recursive filesystem deletion for unmanaged candidate folders. Review the selected relative paths before confirming it. The guard against known imported gallery paths is an additional safety boundary, not a replacement for a backup.

Browser upload is the friendliest choice for routine work. It can upload into an existing gallery or create a child gallery and upload optional photographs in the same side-panel workflow. Select the destination, choose files, and follow the progress display. The application checks the destination and file information before accepting each result.

The upload settings distinguish the ordinary server path from browser-assisted preparation. **Allow all server-supported formats** leaves the picker aligned with the server's detected capabilities. **Prefer phone-rendered JPG/PNG/WebP, no RAW/DNG** asks mobile browsers for a rendered image representation when possible, which is useful when an iPhone ProRAW/DNG workflow produces unsuitable color. The browser hint is not a guaranteed transcoder, so the server still validates the actual uploaded media.

Browser-assisted preparation is enabled independently and is checked by default in the upload workflow. The browser can prepare optimized thumbnails and bounded ZIP batches before sending them. If browser preparation fails before any server-side write starts, ordinary image selections can fall back automatically to the classic server upload path. Administrators can bound the default and maximum browser worker counts, images per batch, ZIP-to-PHP-upload-limit ratio, maximum ZIP batch bytes, and source chunk size used by browser-side thumbnail rebuild. The normalizer also enforces a hard worker cap so an accidental setting cannot create unbounded parallel requests on shared hosting.

Current browser-pipeline defaults are 8 workers, an 80% target of the detected PHP upload limit for each store-only ZIP batch, at most 8 images per ZIP batch, and a 24 MiB absolute ZIP-batch cap. The validated ranges are 1–32 workers, 10–95% of the PHP upload limit, 1–64 images per batch, and 1–128 MiB for the absolute ZIP cap. Browser thumbnail rebuilding uses a 512 MiB source-ZIP chunk by default, clamped to 16 MiB–3 GiB, and the source builder normally caps a chunk at 96 items with an internal hard limit of 512. These are safety ceilings and defaults, not a recommendation to raise a slow shared host to the maximum.

When browser-assisted upload is enabled, the chooser also accepts ZIP archives such as an iCloud Photos export. The browser inspects the archive locally, adds supported JPEG, PNG, WebP, and GIF images to the ordinary preparation queue, and skips folders, hidden metadata, and un-

supported entries. The selected ZIP itself is never uploaded to PHP. Stored and Deflate archives are supported; encrypted, multi-disk, ZIP64, damaged, suspiciously compressed, or excessively large archives are rejected before any server-side upload begins. ZIP import has no classic-PHP fallback, so keep browser-assisted preparation enabled when selecting an archive.

The global automatic-upload rename option can run the same post-scan naming policy used by the Media Renamer after browser, API, or WebDAV ingestion. Leave it off when external filenames are part of your archival convention; enable it when all arrival paths should converge on the gallery's normalized naming scheme.

14.11 Generated metadata and automation

The optional local or worker-based image-analysis queue can record visual metadata separately from the human title, description, tags, and EXIF fields. Administrators can review that metadata in the photo editor and use it as searchable evidence without silently replacing editorial text. Configured OpenAI assistance can generate a photo or gallery description, clean up existing text, summarize context, describe visible content from safe generated thumbnails, or suggest a translation. These actions insert reviewable drafts; they do not publish or save text automatically unless the administrator completes the ordinary save operation.

For repeated transfers, create a scoped upload-automation token for the intended gallery and send supported files to the upload endpoint. Tokens can be revoked independently and must be treated like passwords. The raw API key is shown only when it is created; only its hash is stored afterward, so copy the new key to the intended client at creation time. The global **API manager** lists automation credentials across galleries and provides a central revocation workflow without exposing the original secrets. Gallery-level controls remain available for the same scoped credentials.

The Windows companion uses the same gallery-scoped upload contract. It has two normal upload modes. **Watch folder** ignores files that were already present when watching begins, waits for new files to become stable, remembers confirmed uploads locally, and retries transient failures with backoff. It can optionally attach the current Microsoft Flight Simulator camera latitude/longitude/altitude through SimConnect immediately before each watched upload, and can delete a watched source only after the gallery confirms a successful upload. **Manual upload** accepts a file selection independently of the watch folder and can either generate responsive thumbnails on the PC with worker processes or ask the gallery server to generate them. Thumbnail work and network uploads are pipelined rather than staging an entire large selection first. Tray support lets the watcher or manual job continue while the main window is hidden.

The same Windows application can optionally run a low-priority AI metadata worker. It claims one authenticated analysis job, downloads only the claimed asset, produces internal searchable metadata with the selected local backend/model generation, and returns the result to the existing upload-automation endpoint. This worker is separate from human captions. A gallery-level **Force AI metadata regeneration** action clears the stored generated rows/old queue state for that gallery branch and queues fresh work; the heavy analysis still runs on the Windows worker.

Mobile WebDAV credentials are gallery-scoped and use HTTP Basic authentication plus an unguessable path token. Creating a connection shows its generated password once; the server keeps only a password hash afterward. The Admin page lists the connection label, target gallery, server URL, and last-use time and can revoke the connection. The WebDAV endpoint supports the minimal `OPTIONS`, `PROPFIND`, `MKCOL`, and `PUT` behavior needed by photo-transfer clients such as PhotoSync. Current WebDAV ingestion accepts JPG, PNG, GIF, and WebP. Configure the mobile client to convert HEIC to JPEG before transfer where available. Uploaded media then enters the same scan, optional auto-rename, database, and thumbnail pipeline as other uploads.

15 Thumbnail generation and public rendering

15.1 What a thumbnail is

A thumbnail is a smaller display copy of a photograph. The original photograph may be several thousand pixels wide and occupy many megabytes. A gallery card on a phone may need only a few hundred pixels. Sending the full original for every small card would make visitors wait for detail they cannot see at that size.[12]

Imagine a gallery containing 60 photographs, each with a 12-megabyte original file. Loading every original for the first grid could require hundreds of megabytes. Small thumbnails can reduce that first viewing task dramatically. The original remains available for authorized full viewing or download, while the grid receives efficient previews.[13]

Thumbnails serve several everyday purposes:

- gallery grids appear sooner;
- scrolling uses less memory and network capacity;
- mobile visitors receive an image closer to the size of their screen;
- gallery covers and search results remain compact;
- the large viewer can begin with a suitable preview while a larger source is prepared;
- repeated visits can reuse files already stored in the browser cache.[14]

15.2 Why several thumbnail sizes exist

One small size cannot suit every screen. A narrow phone card may look clear with a 300-pixel thumbnail. A wide desktop card may need 600 or 800 pixels. A high-density screen uses more image pixels to draw the same physical area and may benefit from a larger candidate.

PHP Gallery can prepare common sizes such as 300, 600, 800, 960, 1280, and 1600 pixels, depending on configuration and source-image limits. The number describes the maximum long side of the generated copy. Portrait and landscape photographs keep their original proportions.[15]

The smaller candidates serve grids and covers. Larger candidates serve wide cards, high-density displays, search presentations, map previews, and the large viewer. The browser can select from the available group instead of receiving one fixed size for every situation.

15.3 Why JPEG and WebP are available

JPEG is widely compatible and familiar for photographs. WebP often provides similar visual quality with fewer transferred bytes. PHP Gallery can generate both so a compatible browser

[12] Thumbnail generation keeps the source photograph as the archival and full-viewing asset. The generated copy is a replaceable browsing aid.

[13] Actual savings depend on image content, quality, format, dimensions, and the browser's selected candidate. The example illustrates scale rather than promising one fixed compression ratio.

[14] A browser cache stores recently used web resources on the visitor's device according to server and browser policy. Reuse can make a later visit feel much quicker.

[15] For a landscape photograph, the long side is usually the width. For a portrait photograph, it is usually the height. The shorter side is calculated from the original aspect ratio.

can use WebP while JPEG remains a dependable alternative.[16]

The visitor does not choose a format manually. The page tells the browser which versions exist, and the browser chooses a format it understands. The original photograph keeps its own source format and remains separate from these browsing copies.

15.4 When thumbnails are created

Thumbnails can be prepared when photographs are imported or uploaded, rebuilt through Admin maintenance, or generated through a guarded repair process when the public page discovers that an expected copy is missing. Preparing them near upload time gives later visitors the smoothest experience because the required files already exist before the first visit.[17]

A gallery owner should review thumbnail maintenance after:

- importing a large existing folder tree;
- changing enabled thumbnail sizes or quality;
- restoring a backup that contains originals and database data but an incomplete thumbnail cache;
- moving an installation between servers with different image-format support;
- seeing repeated original-file fallbacks or missing previews in System Health.

Generated thumbnails are reproducible cache data and can be rebuilt from the source photographs. The sources themselves require permanent backup protection.

15.5 DNG and RAW display masters

DNG photographs can remain the archival source while PHP Gallery creates a browser-display master for thumbnail generation and public viewing. The display master is generated derivative data, not a replacement for the original DNG. If the required local conversion tool is unavailable or conversion fails, the Admin upload and thumbnail reports identify the failed DNG derivative. Thumbnail maintenance can rebuild the master later; deleting generated derivatives does not delete the source DNG.

Runtime diagnostics exposes the active DNG conversion policy together with the server's actual GD, Imagick/ImageMagick, and format capabilities. The source policy can use automatic fallback with full RAW decoding first and the embedded camera preview second, explicitly prefer full RAW decode, or prefer the embedded preview. Color handling can force browser-safe sRGB, preserve the decoded appearance as closely as possible, or use camera white balance when available. These choices affect generated browser derivatives only. They never rewrite the archival DNG source, and their availability still depends on the image delegates and memory limits provided by the host.

[16] Compression efficiency varies with the photograph and quality settings. Fine texture, noise, gradients, and sharp graphic details can produce different results.

[17] Large imports can require meaningful processing time and storage. Running preparation as an intentional Admin task gives the owner a clearer opportunity to monitor completion.

15.6 What visitors receive

The gallery sends the browser a list of thumbnail copies that really exist for a card. The browser considers the space available on the page, the sharpness needs of the screen, the supported format, and its own loading decisions. It then requests an appropriate file.

For example, suppose a photograph has 300, 600, 800, and 960-pixel copies:

- a small phone card may receive the 300-pixel copy;
- a medium card may receive the 600-pixel copy;
- a large or high-density card may receive the 800 or 960-pixel copy;
- the large viewer may request a still larger preview prepared for that purpose.

The exact choice can differ between browsers and screens. This flexibility is useful because the server does not need to guess one universal size.

15.7 Responsive browser selection

Responsive browser selection is the default mode under **Admin > Theme > Layout**. The page exposes the available thumbnail candidates immediately and the browser chooses the one that best matches the rendered card and display density. JavaScript is not required for this selection.[18]

Responsive selection usually transfers one chosen thumbnail for each loaded card and is the normal choice when predictable browser-native behavior matters most.

Situation	Likely result
Narrow mobile card	A smaller thumbnail candidate is usually sufficient.
Wide desktop card	The browser may choose a medium or larger candidate.
High-density display	A higher-resolution candidate may be selected for the same CSS size.
JavaScript unavailable	Responsive candidate selection still works.

The 300-pixel copy acts as a fallback when available. Its presence does not force a modern browser to download it first because the larger choices are presented at the same time.

15.8 Progressive thumbnail sharpening

Progressive thumbnail sharpening is an optional mode under **Admin > Theme > Layout**. The gallery first presents a small thumbnail, usually 300 pixels, and later replaces relevant cards with larger copies. On slower connections this can make the collection recognizable earlier, at the cost of potentially transferring both the small and replacement thumbnail.

15.8.1 What happens on the screen

1. The page reserves the final card geometry.

[18] The underlying web standard calls the candidate list a *srcset*. Gallery owners can rely on the visual behavior without learning that markup term.

2. Small thumbnails begin filling visible cards.
3. Relevant cards are queued for a larger candidate.
4. The larger copy loads while the small one remains visible.
5. The card is replaced after the larger copy is ready.

Larger upgrades are queued rather than started for every card at once, and visible cards receive priority over distant content. The current progressive option applies to photo cards inside an open gallery. Gallery covers, collage cells, home-page gallery cards, and Admin images continue using responsive browser selection. The Admin interface labels progressive sharpening as Beta.

15.8.2 The transfer tradeoff

Progressive mode can transfer two files for one photograph: the small first copy and the sharper replacement. Responsive mode often transfers one browser-selected copy. Progressive mode therefore favors earlier visual population, while responsive mode usually reduces total transfer for the same card.

15.9 Which mode should an owner choose?

Choose **Responsive browser selection** for a dependable default, efficient browser-native choice, and typically one thumbnail transfer per loaded card.

Try **Progressive thumbnail sharpening** when early page response matters strongly, many visitors use slow connections, and a small-first visual experience suits the gallery. Test it with representative galleries and devices before making it the normal presentation.

Useful questions include:

- Do visitors see the first photographs sooner?
- Does the small version look acceptable during sharpening?
- How much extra data is transferred on a typical visit?
- Do phones and high-density screens sharpen at a comfortable pace?
- Does the chosen mode suit the audience's likely connection quality?

15.10 Loading priorities and stable layout

The first visible photographs receive earlier loading attention because they shape the visitor's first impression. Photographs farther down the page use lazy loading, which means the browser can wait until they approach the visible area. This avoids spending connection capacity on the bottom of a long gallery while the visitor is still looking at the top.[19]

The gallery knows the width and height of indexed photographs. It uses those proportions to reserve stable card shapes before image loading finishes. Text, buttons, and neighboring cards therefore keep their positions as thumbnails appear.

[19] Lazy loading timing belongs partly to the browser. Browsers consider scrolling, viewport distance, connection state, and their own implementation policy.

Visitors who prefer reduced motion receive a calm presentation without a required continuous loading animation. Photograph links remain usable throughout loading and sharpening.

15.11 Thumbnail quality and storage

Higher quality preserves more fine detail and can create larger files. Lower quality saves space and transfer while introducing more visible compression. The best setting depends on subject matter, screen use, hosting capacity, and audience. Detailed foliage, hair, text, and fine architecture can reveal compression more readily than broad soft backgrounds.

Every additional enabled size and format uses storage because it creates another copy for each photograph. The benefit is more precise delivery to different screens. System Health and storage statistics help an owner understand the resulting cache. A large source archive may produce a substantial thumbnail collection, yet those files remain reproducible from the originals.

15.12 Thumbnail compatibility and maintenance modes

The maintenance panel has two compatibility policies. **Modern** generates WebP thumbnails wherever the server can write WebP. **Legacy** generates JPEG fallback thumbnails as well as WebP. After switching an installation permanently to Modern, **Remove legacy JPG thumbnails** deletes only generated JPEG thumbnail files; originals, WebP derivatives, and DNG display masters are preserved.

A full **Check missing thumbnails** pass inspects imported photographs and records galleries with missing, stale, metadata-missing, or wrong-ratio generated variants without creating anything. The stored result enables **Create missing thumbnails**, which targets only the reported images. **Create all thumbnails** runs the broader creation path. These Admin operations remain the primary deliberate maintenance workflow. Public browsing does not start an unbounded repair sweep, but the current renderer has a small guarded background warmup for specific cards that had to fall back to authorized original media because an expected derivative was missing.

Refresh thumbnail database reconciles database metadata with existing generated files and removes wrong-ratio generated files instead of publishing them. **Delete all thumbnails** is a separately confirmed destructive cache operation; source photographs remain untouched and the derivatives can be rebuilt later.

For hosts where server-side image processing is the bottleneck, an optional **Browser-side thumbnail rebuild** is available and is off by default. The server sends original files to the authenticated browser in bounded source ZIP chunks, the browser generates thumbnail variants, and prepared thumbnail ZIP batches are>uploaded back for server validation and installation. The source-chunk and batch limits are configured with the browser-upload settings so the workflow does not depend on one oversized PHP request.

15.12.1 Guarded public thumbnail warmup

When the server has rendered a publicly authorized photograph through an original-media fallback, it can mark that exact photograph and the missing supported sizes as a background warmup candidate. The browser waits for idle time, stops when the page is hidden, sends at most two candidates per request, makes at most four requests from one page load, and spaces requests by about 1.8 seconds. The server independently caps the normalized submitted list at 24 image IDs, but the actual generation budget is smaller: an anonymous request processes at most two

source images within roughly four seconds, while an authenticated administrator request processes at most four within roughly eight seconds.

A candidate carries an HMAC token tied to the image ID, gallery ID, filename, relative path, and gallery folder that the server originally rendered. Before generating anything, the endpoint reloads the image/gallery, verifies that token, repeats current visitor access checks, normalizes the requested sizes to the configured thumbnail set, consults thumbnail metadata when available, and skips variants that are already renderable. A per-image 60-second cooldown prevents repeated expensive retries. A single global non-waiting worker lock means concurrent public traffic cannot stack several image-decoding jobs; a busy request simply reports that another warmup worker is active. The warmup is enabled by default through the internal `thumbnail_background_warmup_enabled` application setting and writes bounded diagnostic events to the Admin log when logging storage is available.

Security note. Background warmup does not turn a private or otherwise inaccessible photograph into a public repair target. The signed candidate proves that the server rendered that image as a fallback candidate, and the endpoint still repeats the current access policy before opening the source or writing derivatives.

15.13 If a thumbnail is missing

A missing thumbnail may appear as a slower original-file fallback, an empty preview, or a maintenance warning, depending on the image and available files. Open the thumbnail maintenance tools, inspect the affected gallery or sizes, and rebuild the missing variants. Review write permissions and image-format support when rebuilding repeatedly fails.

After repair, open the gallery with an empty browser cache and confirm that covers, cards, and the large viewer receive the expected previews.

16 Lightbox, maps, EXIF, and voting

16.1 Asynchronous lightbox metadata

The lightbox is the large photograph viewer that opens above the gallery page. It lets visitors concentrate on one photograph while retaining forward, backward, close, full-screen, slideshow, map, and other available controls.

The gallery prepares the viewer when it becomes useful and obtains information for nearby photographs in manageable groups. A visitor can therefore move through a large gallery without requiring the first page to carry every title, preview, map point, and vote at once.[20]

The Left and Right arrow keys move one photograph backward or forward. `Shift+Left` and `Shift+Right` move ten positions for faster travel through a large result set. The jump follows the same stable ordered sequence as ordinary navigation, wraps at the ends, and also works when the active result set is a Smart Gallery. The translated keyboard-help text lists these shortcuts; no separate graphical jump buttons are added.

The viewer usually starts with an appropriately sized preview. A larger or original source can be used where the action and access policy call for it. Nearby previews may be prepared quietly so the next photograph appears smoothly.

16.2 Zooming and panning a photograph

The lightbox zoom range is 100–400%. Use the minus and plus controls, or select the percentage control to return to 100%. Keyboard shortcuts are `+` or `=` for zoom in, `-` or `_` for zoom out, and `0` for reset. Focus indicators and the current percentage remain exposed to assistive technology.

Mouse-wheel and trackpad zoom follows the pointer: the part of the photograph under the cursor stays in place as the scale changes, including during several rapid zoom steps. Pinch zoom uses the gesture midpoint. Once enlarged, the photograph can be dragged horizontally and vertically. At 100% on touch devices, the normal horizontal swipe continues to change photographs. Control/Command-modified wheel input remains available to the browser for page zoom.

At 100%, the photograph is fitted and centered in the viewer. Full-screen mode uses the same zoom and pan behavior, and its Close and other controls remain clickable while the image is enlarged.

Zooming above 100% immediately requests the protected full-quality source for the active photograph. The existing preview remains visible until the larger image is ready, and the current zoom and pan position are preserved. In full screen and mobile full screen, this transfer shows a progress bar with a percentage and a byte count; the normal (non-full-screen) viewer keeps the existing compact loading indicator instead. Changing photographs, closing the viewer, or starting a newer full-quality request cancels the previous transfer. A failed or cancelled full-quality request leaves the protected preview in place.

Navigation, slideshow start, close, and reopen return the viewer to a centered 100%. Entering or leaving full screen keeps the current scale when the photograph itself has not changed and adjusts pan to the new viewport. Reduced-motion preferences remove the short discrete zoom transition.

Zoom affects presentation only. It does not alter the stored photograph or bypass media authorization. Password, share-link, NSFW, Smart Gallery, map, voting, pagination, and lightbox metadata rules continue to apply.

[20] The current viewer normally requests information in nearby groups of photographs. Access checks are repeated for each request so private gallery rules continue to apply.

16.3 EXIF and map policy

EXIF is information many cameras and phones save inside a photograph. It can include capture date, camera, lens, exposure, dimensions, orientation, and GPS coordinates. PHP Gallery can use this information for date suggestions, maps, technical details, ordering assistance, and duplicate review.

Maps load when a visitor asks to see them, which saves work for visitors interested only in the photographs. A gallery can show individual photo locations and an overview of available points. Parent and child settings determine which galleries expose this information. The global EXIF/GPS public-display setting is the fallback for galleries whose override is set to inherit. Admin also shows how many galleries currently have an explicit override and can reset all of those overrides back to inherit in one confirmed settings save. The reset is schema-guarded: if the override state cannot be verified, the application refuses the change and directs the administrator to System Health instead of guessing.

Selecting **Open photo** in a gallery-map marker opens that photograph in the current lightbox when it belongs to the active gallery. This also works when the photograph is outside the currently displayed pagination page: the viewer requests only a bounded metadata window around the selected photograph. In full-screen split-map mode the map stays open while the photograph pane changes. From a normal map overlay, the map closes and reveals the selected photograph in the viewer. If the enhanced viewer cannot resolve the selection, the same control follows the canonical photograph page; it does not link directly to the raw image file.

> **Security note.** GPS data can reveal sensitive locations. Administrators should review inherited map settings, image metadata, and public access before publishing personal photographs.

16.4 Flight maps and navigation data

Flight-oriented collections can use the optional SimBrief integration to retrieve the latest operational flight plan for a Pilot ID or pilot name. The gallery editor saves the OFP with the gallery, creates an editable Markdown description draft, and writes the OFP route coordinates into the gallery flight-map record. If both identifiers are provided, the Pilot ID is preferred. A failed SimBrief request leaves the ordinary description field and manual route editor available.

Manual route text is resolved offline-first. Imported OurAirports database rows are checked first, then the small bundled CSV fallback. A valid cached provider result may be used next; when an administrator has linked a configured Navigraph account and remote lookup is allowed, an unknown identifier can be resolved through Navigraph and cached under the known AIRAC cycle. The Navigation Data manager shows local database and bundled counts, provides a lookup tester for identifiers such as airports/navaids/fixes, and can refresh the local OurAirports airport/navaid dataset. Navigraph OAuth/account storage is optional and can be disconnected without removing local data.

Public map rendering never calls SimBrief, Navigraph, or another live planning service. It renders only coordinates already stored with the gallery. Saved SimBrief OFP coordinates therefore remain usable even if the remote provider is later unavailable. Manual route text can also bypass identifier lookup completely with explicit `NAME@latitude,longitude` points.

16.5 Voting

Public cards and lightbox controls share image-vote state. The current public vote is a reversible positive vote: pressing the active up-vote again removes it. The displayed score is the sum of stored image votes; there is no separate whole-gallery voting endpoint in the current implementation. Logged-in votes are associated with the account, while anonymous votes use the application's privacy-preserving visitor hash. New positive votes are rate-limited; revoking an existing vote is allowed immediately. Server-rendered forms provide direct behavior, while JavaScript submits normal interactions asynchronously and synchronizes the score across matching views. The server validates image identity, gallery policy, access, schema readiness, CSRF, and request integrity.

16.6 Picture Game

Picture Game is an optional public comparison mode. An administrator enables it for a gallery branch; eligible public photographs from that gallery and its descendants can then participate subject to the ordinary access and visibility rules. The public game shows two photographs side by side, records which pair was presented, and lets the visitor choose the preferred image before moving to the next comparison. The gallery can also present leading results accumulated by the game.

Because presenting a new pair creates persistent game state even before the visitor chooses a winner, Picture Game requires its database schema to be known and available. If the required storage cannot be verified, the write is refused rather than silently falling back to an untracked comparison. The global feature switch can hide Picture Game without deleting existing game data or per-gallery enable settings.

17 Duplicate Photo Detector

The Admin Duplicate Photo Detector analyzes a selected gallery branch by default and can expand to global scope when the administrator enables the corresponding option. It reports exact and possible duplicate pairs, provides gallery and photo context links, records review decisions, and delegates confirmed deletion to the normal gallery image-deletion service.[21]

17.1 Evidence categories

17.1.1 Exact and possible findings

An exact duplicate contains the same file content. The application recognizes this through a checksum, which acts like a very detailed digital fingerprint calculated from the file. Matching fingerprints provide strong evidence that two files contain the same bytes.

A possible duplicate has supporting similarities without an exact fingerprint match. The detector can compare file size and available camera information such as dimensions, capture time, camera, lens, exposure, aperture, focal length, ISO, and orientation. Two separately exported copies of one photograph may differ as files while retaining enough shared information to deserve review.

Missing camera information simply gives the detector fewer clues. The results remain suggestions for a person to inspect. Open both photograph links, compare their gallery context and quality, and delete only the copy whose removal fits the collection.

17.2 Persistent review ledger

The ledger belongs to the authenticated administrator. Pair entries store canonical image identifiers, with the lower identifier in the low column and the higher identifier in the high column. Gallery entries apply to the exact stored gallery. Parent and child gallery decisions remain independent.

Available review actions include:

- ignore the selected pair;
- ignore all findings from the left image's exact gallery;
- ignore all findings from the right image's exact gallery;
- clear the current administrator's ledger;
- delete a confirmed duplicate through validated normal image deletion.

AJAX actions refresh the detector fragment in the existing Admin right-side panel. Direct POST requests use redirect behavior as a fallback. Server-owned scan jobs preserve scope between continuation requests, and deletion prunes the affected image from current job results.

[21] The detector remains a review tool. Evidence supports administrator judgment because matching file size and photographic metadata can describe distinct photographs under some workflows.

18 Access control and security

18.1 Layered access evaluation

Privacy is evaluated before protected output is assembled. Gallery visibility and inherited access rules are combined with per-photo visibility, password/share-link state, and age-restriction rules where configured. The same authorization applies to gallery pages, thumbnails, previews, original files, lightbox metadata, maps, search results, and downloads.

Admin operations require an authenticated account. Forms include a private request token, and the server validates the target gallery, photograph, and requested operation before changing persistent data.[22]

18.2 Passwords and share links

Protected galleries store password hashes and grant session-scoped access after successful verification. Share links use server-generated token state and expiry policy. Password reset uses one-time hashed tokens, an optional recovery email, configured lifetime, and the selected mail transport. Use HTTPS, strong administrator credentials, restricted database accounts, and protected mail/API secrets.

18.3 Viewer accounts and invite-only registration

Viewer accounts are an optional identity layer for personalized gallery features. They are separate from the administrator account and, by themselves, do not unlock a private or password-protected gallery. Existing gallery passwords, share links, Smart Gallery access, and media authorization continue to decide what a visitor may see.

The complete viewer-account subsystem is wrapped by a master switch in **Admin > Features**. This master feature is disabled by default. While it is off, the public gallery shows no viewer Login/Account controls, viewer routes are unavailable, and the **Viewer accounts** entry is hidden from the Admin Account menu. Existing viewer identities, favourites, and private collections remain stored; the switch controls reachability rather than deleting data.

After enabling the master feature, the administrator can open **Account > Viewer accounts**. That page retains the separate **Viewer accounts (invite-only)** mode switch. The mode switch controls viewer login/invitations inside the enabled subsystem, while the master Features switch controls whether the subsystem is exposed at all.

An administrator manages viewer identities from **Account > Viewer accounts**. The existing invite-only workflow remains available: the administrator may optionally bind an invitation to an intended email address and copy its show-once secret link. Opening the link does not create an account. The recipient submits an email address, receives a verification message through the configured mail transport, confirms the verification in the browser, and then chooses a password of at least 15 characters. There is no general **Register** or **Sign up** page.

The same Admin page can also create a viewer account directly. Enter the viewer email and either provide a temporary password that meets the viewer password policy or leave the password blank so the gallery generates a strong one. The temporary password is shown once after creation. If the optional account-created notification is sent, the message contains the trusted login address

[22] The technical name for the private form token is a CSRF token. Gallery owners normally encounter it only through ordinary Admin forms.

and instructions but never the temporary password; provide that password separately through a trusted channel. With the master feature enabled, direct account creation remains available while the subordinate viewer frontend mode is disabled so accounts can be prepared in advance, but those viewers cannot sign in until that viewer mode is enabled.

A directly created viewer cannot use the temporary password as a durable login. After the first successful credential check, the browser is sent to the private first-login page and must choose a different compliant password. Until that replacement succeeds, the gallery does not establish the normal viewer identity, issue **Remember me**, or allow viewer favourites/collections authority. The ordinary forgotten-password reset can also replace the temporary credential through its independently verified recovery flow. A simultaneous administrator login in the same browser remains independent.

Existing viewer accounts are listed on the Admin page with their account state and security controls. An active viewer can be suspended with **Suspend** or signed out everywhere; a suspended viewer can be reactivated with **Restore**. Suspension blocks viewer authentication and invalidates the viewer's existing session, remember-me, reset, pending verification/email-change, and prepared collection-share authority through the central viewer lifecycle service. Restoration makes the account active again but does not revive any credential or session that existed before suspension, so the viewer must authenticate again normally. A forced first-login password replacement remains required across suspend/restore. **Sign out everywhere** leaves an active account active while revoking its viewer login authority on all devices. These security controls remain available to the administrator while the master feature is enabled but the subordinate viewer frontend mode is disabled. Turning the master feature off hides the viewer-management surface as well. None of these switches or security controls removes favourites, private collections, collection items, photographs, galleries, Smart Galleries, gallery share links, or administrator authentication.

Admin deletion remains a separate destructive action with explicit confirmation. It removes the viewer identity and viewer-owned authentication, lifecycle, favourite, and collection state through the prepared database relationships. It does not delete photographs, galleries, Smart Galleries, gallery share links, or administrator accounts/sessions.

The public header shows **Login** only while viewer accounts are enabled. After sign-in it shows **Account**. The account page shows the verified email, links to the private **Favourites** and **Collections** pages, **Change password**, **Change email**, **Delete account**, and a viewer-only **Log out** action. Logging out a viewer or changing viewer account authority does not log out an administrator who happens to be authenticated in the same browser session.

A signed-in viewer can mark an authorized photograph as a favourite from the small heart control on a gallery card or in the lightbox. The same photograph is kept synchronized between the card and lightbox, and the private **Favourites** page lists saved photographs that the viewer is still allowed to open. Removing a favourite deletes only the viewer's saved reference; it does not delete or modify the photograph.

A favourite is never an access grant. If the source gallery later becomes private, its password/share permission expires, or the photo is otherwise no longer authorized for that request, the saved favourite does not bypass those rules and the source metadata is not shown on the Favourites page. A browser that is simultaneously signed in as administrator and viewer still needs the viewer's independent source authorization before the viewer can save that image as a favourite.

A signed-in viewer can also create private named **Collections**. A collection is an ordered list of references to existing photographs; it is not a gallery, does not copy photographs, and does not change the administrator-owned gallery hierarchy. From an authorized gallery card, Smart Gallery card, or the private **Favourites** page, the viewer can use **Add to collection** and choose one of their own collections. The private Collections area supports creating and renaming a collection, removing

photographs, moving visible photographs up or down, and deleting the collection itself.

Collection ownership and photograph access are separate decisions. Saving a photograph stores only its internal image reference and order. The collection does not store a gallery password, gallery share token, filesystem path, thumbnail path, or a remembered access decision. Each time the collection is opened, the gallery re-evaluates the current source-gallery and photo authorization. If a previously saved photograph is no longer authorized, it is omitted without exposing its title, filename, gallery title, thumbnail, EXIF data, or reason for denial. The saved reference can remain so the photo becomes visible again if normal source access later returns.

This rule also applies when the same browser is signed in as both administrator and viewer. Administrator visibility does not become viewer collection authority, and a collection can never be used as a second route around gallery passwords, share grants, age restrictions, or media-serving checks. Favourites and collections are independent: a photo may be in either, both, or neither, and deleting a collection never deletes a favourite or the underlying photograph.

A viewer may create one unlisted, read-only share link from an owned collection. The link expires after 30 days and can be replaced or revoked at any time. The complete secret link is shown only immediately after creation or replacement; later the collection page shows only that a share is active plus its creation and expiry times. Replacing a share makes the previous link unusable. Revoking a share removes only the sharing authority and does not delete the collection, its items, favourites, or photographs.

A recipient does not need a viewer account. Opening the secret link validates it, establishes only a narrow session permission for that one collection, and redirects to a clean address without the secret. The link is reusable until expiry/revocation, so automatic mail or chat link scanners do not consume it. The shared page is unlisted, non-cacheable, and marked not for search indexing.

The share grants access only to the collection container. It never unlocks a source gallery or photo. On every shared view, each stored photo reference is checked again against the recipient's current source-gallery authorization. A password-protected source remains hidden until that recipient independently unlocks it through the normal gallery mechanism; an independent existing gallery-share permission may likewise authorize it normally. Even when the same browser is also signed in as administrator, administrator bypass is not used to populate the shared collection. Direct thumbnail/media routes continue to enforce their own existing source rules. Inaccessible items are omitted without exposing their hidden title, filename, path, gallery, or reason.

Suspending the collection owner revokes active collection shares, and restoring the account does not revive them. **Sign out everywhere** only revokes viewer login authority and does not revoke an otherwise valid share. Deleting the owner or collection invalidates its share without deleting source photographs. The global Viewer Accounts feature switch also gates collection sharing and remains disabled by default.

Viewer **Remember me** uses a dedicated rotating viewer credential rather than the administrator persistent-login token. Restoring that credential signs in only the viewer and does not count as recent password reauthentication. Forgotten-password requests return a generic response and use a two-step reset confirmation so automated link scanners cannot reset a password by opening a URL.

Sensitive account changes require recent viewer password proof. A viewer restored only through **Remember me** is therefore asked to confirm the current viewer password before changing the password, starting an email-address change, or deleting the account. The return path is limited to those account actions and cannot redirect to an arbitrary page.

Changing the viewer password uses the same viewer password policy as activation and reset. A successful change invalidates the old viewer password authority, viewer sessions, and viewer

remember credentials according to the security lifecycle and then asks the viewer to sign in again. Saved favourites remain attached to the same viewer account. An administrator signed in through the same browser session remains signed in.

Changing the viewer email is staged. Submitting a new address does not replace the current verified address. The gallery first sends a verification link to the candidate address within the viewer mail-abuse limits. Opening that link only validates it and prepares a short-lived confirmation state; it does not change the account. The final change occurs only after the viewer submits the separate protected confirmation form. A successful switch signs the viewer out so the new verified email can be used for the next login.

Delete account is a permanent viewer-only action. After recent password confirmation, the viewer must explicitly confirm deletion. The operation removes the viewer account and viewer-owned data such as favourites and private collections according to the prepared database lifecycle. It does not delete gallery photographs, galleries, Smart Galleries, or their existing share links, and it does not destroy a simultaneous administrator login.

Viewer security and private account/favourite/collection/lifecycle pages are served as private, non-cacheable responses. Invitation, verification, reset, and email-change links use the configured trusted application origin. Viewer verification, reset, and email-change mail remains subject to the viewer address, visitor, and global abuse budgets before the configured PHP mail or SMTP transport is used.

Unlisted read-only collection sharing is available only inside the Viewer Accounts feature. Public collection discovery, public viewer profiles, recipient editing/collaboration, viewer uploads/comments, open registration, viewer social login, TOTP, passkeys, and magic-link login remain unavailable.

18.4 Administrator account and mail settings

The account area supports username and password changes, an optional recovery address, password-reset lifetime, and a test message. Recovery mail can use PHP `mail()` or an authenticated SMTP transport with host, port, encryption, username, and password settings. Treat SMTP credentials like database credentials and never include them in exports or diagnostic screenshots.

Google login is optional. The administrator configures the OAuth client and authorized callback/redirect URI, then links a Google identity only while already authenticated with the normal administrator password session. An unlinked Google identity is rejected at the login page and is not allowed to create or claim an administrator profile. After linking, only that stored Google identity can sign in through the Google button for the profile. Disconnecting the link requires the current administrator password. External login does not replace password login; if OAuth configuration or identity-link storage is missing or cannot be verified, ordinary password authentication remains independent.

The login form accepts the administrator username or the configured recovery email. Authentication attempts are throttled by both visitor and normalized submitted identifier when the rate-limit schema is available. The throttle table stores hashed subjects rather than raw IP addresses or raw submitted identifiers. Current policy limits repeated failed logins within a 15-minute window and password-reset requests within longer bounded windows; successful login clears the relevant login throttle state.

Password recovery deliberately returns a generic request result so the public form does not confirm whether a submitted username/email exists. Reset links use one-time hashed token storage and the configured 15-to-1440-minute lifetime. Completing a reset changes the password, marks the token used, and revokes persistent-login tokens for that account. The Account page can send

a test recovery email through the configured PHP `mail()` or SMTP transport.

The **Keep me signed in** option uses a hashed browser token so an administrator can survive ordinary shared-host PHP session cleanup without storing the raw persistent credential in the database. Persistent login remains optional. It is revoked by the appropriate account/logout security workflows and can be unavailable while its schema state cannot be verified, without disabling the current ordinary PHP session.

18.5 Database readiness contract

Features that depend on database schema use one readiness contract. The important distinction is between a schema that was successfully inspected and found to be older, and a schema that cannot currently be inspected.

State	Meaning	Application behavior	Owner action
Available	Required storage was found and checked.	Normal reads and writes for the capability.	None.
Missing	Inspection succeeded, but a required object is absent.	A documented older-schema path may remain available; otherwise the feature asks for a migration.	Back up and apply the pending migration.
Unknown	The schema could not be inspected reliably.	Protected reads may fail closed or optional read-only output may be omitted. Dependent writes are blocked.	Repair database connectivity, selected schema, or metadata permissions.
Disabled	An optional feature is turned off.	The feature is not used.	None unless the feature should be enabled.

System Health and **Runtime Diagnostics** use these states consistently. A **Missing** result normally points to migration work; **Unknown** is an operational problem and should not be treated as proof that a table or column never existed. Diagnostics may show the capability name, status, validated database-object names, suggested checks, and a request reference. They exclude SQL text, raw database exceptions, credentials, tokens, cookies, and private filesystem paths.

18.6 Security-sensitive exceptions

A few capabilities have stricter or more specific behavior:

NSFW Guard. If inspection confirms the older schema without NSFW fields, the site retains its pre-feature behavior until migration. If the fields are **Unknown**, public requests that could expose age-restricted media fail closed and NSFW editing is paused.

Gallery access. A database is treated as the older pre-protection format only when inspection succeeds and all core access fields are absent. A mixed layout is an incomplete migration, so protected routes remain unavailable until it is repaired. The older publication value

`draft` is written only when the field definition is known to require it; current schemas use `unpublished`.

Share-link revocation. Revocation may use the smaller known field set needed to clear the validating hash because that operation reduces access. It still stops when its own required fields are **Unknown**.

Persistent Admin login. A missing remember-login table disables persistent browser tokens but does not prevent an ordinary password login from creating the current PHP session.

Password reset. Recovery requires both the recovery-email field and reset-token storage. Missing storage produces migration guidance; **Unknown** storage produces a temporary failure instead of an incorrect account/token result.

Google identity links. External login requires both valid OAuth configuration and known identity-link storage. If that storage is missing or unknown, password login remains available independently.

18.7 Writes, maintenance, and credentials

Operations that change files, credentials, or persistent records perform their readiness check before the first target-changing step. This includes gallery/photo mutations, uploads, WebDAV writes, gallery migration, thumbnail maintenance, database repair/cleanup, and application update activation. An **Unknown** result stops the operation before the target is changed. A confirmed **Missing** result may use an older-schema compatibility path or migration bootstrap only where that behavior is documented.

Credential revocation follows the same restrictive principle as share-link revocation: it may use a smaller known field set when those fields are sufficient to disable access. Creating or trusting a credential still requires the complete storage used by that authentication path.

When a write is blocked as **Unknown**, repair database inspection before retrying. When the state becomes **Missing**, back up and apply the required migration. When it becomes **Available**, retry the operation once and check both filesystem and database results. Resume an existing recoverable migration/update job rather than starting a duplicate job.

18.8 Optional features

Optional presentation features can degrade without disabling the main gallery when doing so cannot reveal protected content or grant access. A map may be omitted if its optional storage cannot be read, for example, but saving a GPS/lightbox override still requires the relevant field to be known.

Voting and Picture Game both write persistent state. Picture Game records a displayed pair before a winner is chosen, so loading a new pair is also a write boundary and requires known storage. AI queues, telemetry maintenance, navigation tokens, saved SimBrief route data, and similar optional writes follow the same rule. A feature may return a read-only result without optional persistence only where the interface makes that limitation clear.

18.9 Operational security

Recommended practices include:

- Serve the application through HTTPS.
- Restrict database credentials to the application schema and required operations.
- Keep `config.php`, backups, caches, and generated archives outside public discovery wherever the hosting layout permits.
- Apply migrations and updates through authenticated maintenance workflows.
- Review audit logs and security diagnostics.
- Test protected galleries through anonymous visitor sessions.
- Maintain coordinated filesystem and database backups.

19 Theme, presentation, and localization

The Theme area controls site identity, colors, dimensions, page width, backgrounds, branding, favicon assets, language, gallery-card presentation, lightbox defaults, GPS presentation, favorite navigation, and public thumbnail rendering. Settings use normalized scalar values or focused service models.

The Theme editor does not use a separate light/dark-mode switch. Instead, the saved color controls and an optional Custom CSS skin define the public visual palette directly. It also sets gallery-card and subgallery layouts and can configure up to three header shortcuts. Each shortcut can target the main page, one gallery, or remain empty. Anonymous visitors see only configured gallery shortcuts that are still public and listed; duplicate or deleted gallery selections are discarded when the settings are saved. Leaving all three slots empty hides those favorite-gallery buttons. Gallery cards may also show a configurable contained-picture count badge. When that badge is effective for a gallery, opening the gallery keeps the same branch count visible in the hero panel, counting images in the current gallery and its accessible subgalleries under the same public/access rules as the card. Per-gallery presentation can inherit the site defaults or override the choices offered by the gallery editor.

19.1 Theme control reference

The current Theme page is divided into **Appearance**, **Branding & media**, **Layout**, **Language**, and **Custom CSS**. The following reference records the owner-visible controls and the normalization enforced by the current build.

Area	Control	Current behavior
Appearance	Site identity and colors	Site name plus accent, dark-accent, page-background, panel, open-gallery panel, header-title, and gallery-title colors. The “dark accent” is a hover/outline color, not a dark-mode selector.
Appearance	Corners and font	Corner radius is 0–32 pixels. Font mode is **Classic serif** or **Clean sans-serif**. The live preview updates while these controls are edited.
Appearance	Public page width	**Default**, **Wider**, **Full width**, or **Custom**. Custom width is clamped to 1024–2048 pixels and defaults to 1440 when no valid value exists.

Area	Control	Current behavior
Appearance	GPS pin	When the GPS-maps feature is enabled, photo-card pins and their background underlay can be shown or hidden separately. Pin size is 14–48 pixels with 26 as the default. The underlay slider is displayed as 0–48 pixels, but the current server normalizer treats 0 or another non-positive value as unset and restores 22 pixels; use the separate underlay checkbox to hide the background.
Appearance	Public tag pages	Dedicated tag-page columns and rows use the same safe maxima as other public grids: 1–12 columns and 1–50 rows. Tag-result cards can use the vertical or horizontal gallery-card design independently from ordinary gallery listings.
Appearance	Gallery hero tags	Sort by usage or alphabetically. Either show all immediately or show 1–200 tags before the in-place expansion control; the default threshold is 20. Optional scrolling uses 1–12 visible rows, default 5, and activates only when the rendered tags actually wrap beyond the configured rows.
Layout	Header shortcuts	Three ordered slots. A slot can be empty, point to the main page, or point to a gallery found by the gallery picker. Public visitors see only public/listed gallery targets.
Layout	Gallery-card design	Global default is **Vertical system** or **Horizontal system**. A physical gallery can inherit or override this default where the presentation schema is available.
Layout	Picture-count badge	Global contained-picture badge is enabled by default. Individual galleries can inherit it or save an override. When enabled for the opened gallery, the hero panel also shows the current gallery's branch image count, including accessible subgalleries.

Area	Control	Current behavior
Layout	Public thumbnail renderer	**Responsive browser selection** is the default; **Progressive thumbnail sharpening** is the optional beta mode for photo cards inside an open gallery. Gallery covers and collages continue to use responsive browser selection.
Layout	Pagination	Global pagination is off by default. Columns default to 3 and are clamped to 1–12; rows default to 3 and are clamped to 1–50. Effective items per page are columns multiplied by rows.
Layout	Main-page grid	The front page has its own 1–12-column and 1–50-row grid. This does not change grids inside galleries.
Layout	Per-gallery grid reset	**Reset all custom gallery grids** clears database overrides, restores default subgallery-inheritance state, and removes matching grid keys from `gallery.json` sidecars so a later discovery pass cannot re-import stale grid settings.
Layout	Lightbox default	**Single image**, **Picture strip**, or **3D carousel** while the lightbox-modes feature is enabled. The classic single-image mode is the fallback and the only mode exposed when that feature is disabled.
Branding	Public header banner	Optional Theme-level JPG/PNG/GIF/WebP banner, maximum 8 MB. It replaces the visible site title when no gallery-specific banner is configured.
Branding	Header separator	Optional JPG/PNG/GIF/WebP separator, maximum 8 MB. Width 0 follows the responsive page width; a positive width is clamped to 160–3840 pixels. Height defaults to 72 and is clamped to 8–512 pixels. Normal mode preserves aspect ratio and treats height as a maximum; stretch mode uses the configured render box exactly and can distort the image intentionally.
Branding	Favicon	Upload JPG/PNG/GIF/WebP, drag a square crop, and zoom the crop from 1–3 times. Saving produces 32-, 48-, and 180-pixel PNG favicon variants.

Area	Control	Current behavior
Branding	Theme background	Preserve the original upload and optionally generate a WebP derivative when GD/WebP support is available. The optimized longest side is 1024–3840 pixels, default 1920, with 128-pixel slider steps. It can be regenerated or removed without deleting the original.
Branding	Background visibility/fallback	Background visibility is 0–100 percent, default 65. The gallery fallback source can be none, a new Theme upload, an existing gallery image, or a collage generated from public galleries. Separate actions reset all per-gallery background selections, remove the Theme background, or remove the favicon.
Custom CSS	Preset skin or upload	Presets are the CSS files currently present in `custom_css/`; selecting one copies it to `public/assets/custom.css`. A manually uploaded `.css` file replaces that active file. The active custom stylesheet loads after built-in CSS and saved Theme controls.
Custom CSS	Reset actions	**Reset custom CSS** deletes the active custom stylesheet and clears its preset marker. **Reset to CSS** clears saved palette/font/radius/background-source/GPS visual overrides so the underlying CSS becomes authoritative again; structural page-width settings are intentionally preserved.
Language	Admin/public language	Admin language is stored for the administrator's session/browser; the public language is the site-wide anonymous default. The viewer selector is an independent visitor override, described below.
Language	Pack maintenance	Supported packs show string counts and coverage. The JSON editor validates an object containing string values, imports/exports JSON, and can clear authenticated missing-key diagnostics. English remains the source/fallback catalog.

19.2 Lightbox browsing modes

The Theme editor defines the default public lightbox presentation. **Single image** keeps the classic viewer. **Picture strip** adds nearby thumbnail previews below the active photograph. **3D carousel** places a small set of neighboring photographs behind the active one to provide more visual context while browsing.

A physical gallery can inherit the Theme default or save its own lightbox-mode override. Smart Gallery presentation can also resolve an effective lightbox mode from its own presentation settings and the site defaults. The modes share the same authorized lightbox metadata and navigation pipeline, so fullscreen, slideshow, zoom, maps, voting, and progressive quality behavior continue to follow their ordinary rules. Disabling the global lightbox-modes feature switch hides these browsing controls without changing stored photographs.

19.3 Choosing the interface language

PHP Gallery treats the language of the program interface separately from the text that you write yourself. Changing the interface language changes buttons, menu items, help text, status messages, dialogs, and other application wording. Gallery and photo titles/descriptions can have optional maintained-language versions, but they are edited and saved separately; tags and other owner content remain unchanged.

Open **Theme > Language**. Two independent selectors are available:

- **Admin interface language** changes the language seen by the administrator in the current browser.
- **Public visitor language** sets the default interface language for public pages.

The optional **Viewer language selector** lets individual visitors override that public default in their own browser. The administrator chooses which maintained languages appear in the public selector; disabling the feature returns visitors to the site-wide public language. Admin and public languages do not have to match.

Public pages use a compact language control with native-language labels and locally served flag icons. Choosing a language keeps the visitor on the same public page and remembers the choice for later visits in that browser. **Use site default** removes the personal override. The equivalent `?lang=` parameter remains available for direct links and no-JavaScript use.

19.3.1 Customizing the viewer selector

The selector has five presets: **Classic**, **Solid pills**, **Outline**, **Soft cards**, and **Minimal**. **Settings > General** provides the basic preset/flag choices, while **Theme > Language** contains the detailed design controls and live preview. These appearance settings do not change the offered languages, site default, Admin language, or a visitor's personal choice.

19.3.2 Multilingual gallery and photo text

Gallery and photo editors keep the existing title and description as source content. Use **Language of the current title and description** to classify that text as English, Czech, German, Swedish, or leave it **Not specified**. Existing installations are not automatically classified as English.

Open **Other languages** with the globe control to enter optional translated titles and descriptions. Gallery titles and descriptions fall back independently. A saved photo-language row is instead treated as one caption variant: blank fields remain hidden rather than mixing source-language text underneath translated photo text. Leaving both translated fields blank removes that language row. Saving occurs through the normal gallery/photo form and remains in the existing Admin side panel.

Public pages use the viewer's selected maintained language. Matching text is shown on cards, gallery descriptions, photo overlays, lightbox metadata, search results, alt text, and SEO metadata. Missing gallery fields fall back to source content; a present photo caption variant displays only its saved translated fields. Translations do not change URLs, slugs, folders, filenames, ordering, visibility, passwords, NSFW rules, or media authorization.

If OpenAI text assistance is configured, **Suggest translation with OpenAI** sends the source field and inserts a reviewable draft in the target field. The draft is never saved or published automatically. Manual translation remains fully available without a provider.

19.3.3 Available languages and their current state

English is the canonical source language, the new-installation default, and the fallback language. Czech, German, and Swedish are also maintained as complete catalogs. These four are the currently selectable interface languages.

Language	Code	Current role
English	`en`	Complete canonical source, new-installation default, and fallback language.
Čeština	`cs`	Complete maintained selectable Czech translation.
Deutsch	`de`	Complete maintained selectable German translation.
Svenska	`sv`	Complete maintained selectable Swedish translation.

Important. Only English, Czech, German, and Swedish are currently selectable. English remains the fallback if a maintained translation is missing.

19.3.4 How translation packs work

A translation pack is simply a dictionary of named interface messages. The program asks for a stable message key such as "save gallery" or "delete selected photos", and the active language pack supplies the wording that should be shown. The internal key stays the same in every language; only the displayed text changes.

The **Detected language packs** table shows coverage for the supported languages, and the **Diagnostics** subsection records missing keys observed during an authenticated Admin session. English supplies fallback wording when a maintained translation lacks a key.

The **Pack editor** is intended for careful maintenance or for importing a prepared translation. Language data is stored as JSON. In that editor, the text on the left side of each JSON pair is the stable key and should not be translated. The text on the right side is the value that may be translated. Placeholders such as `\{count\}`, `\{gallery\}`, or `\{time\}` represent live values inserted by PHP Gallery and must keep the same names in every language.

A language pack can also be exported as JSON, translated outside PHP Gallery, and imported again. Imports are accepted only when the file is a JSON object containing text values, so a translator can work on the pack without receiving access to the gallery server.

For developers and maintainers, the maintained catalogs live under `app/lang/`. JSON is the canonical editable format. English, Czech, German, and Swedish PHP dictionaries are retained only as compatibility fallbacks for installations that do not have the corresponding JSON file. Server-rendered pages and browser JavaScript use the same active-language plus English-fallback model, so a translation does not need to be duplicated in separate front-end and back-end catalogs.

Custom CSS supports installation-specific visual refinement. Theme-generated CSS exposes validated variables and computed selectors through a dedicated route. Gallery branding can override selected site-level assets while preserving effective fallback behavior.

19.4 Gallery hero tags

The **Appearance > Gallery tags** subsection controls the compact tag collection below the title of an open gallery. The tag area uses the complete width of the hero panel, so tags can continue wrapping toward the right edge instead of being limited by the normal readable paragraph width. Direct gallery tags and tags collected from contained content remain separate semantic groups, but each group follows the same global display policy.

The same subsection also controls the public tag landing page. **Galleries per row** and **Rows per page** define the tag-result gallery grid and its page capacity, while **Gallery-card design** selects the vertical or horizontal card presentation for those results. These values affect public pages opened through a tag; they do not change ordinary gallery pages. If the dedicated values have not been saved, the global Theme grid and gallery-card design remain the fallback. The global pagination switch still determines whether a long tag result is split into pages. From **Edit tags**, **Configure tag display** opens this subsection directly, and **Manage tag metadata** provides the reverse contextual link.

By default, the browser initially shows 20 tags. The limit can be changed from 1 to 200 with either a range slider or an exact number field. When more tags exist, **Display all tags** appears and expands the existing tag elements in place with JavaScript; no page reload or additional server request is used. The same control can collapse the collection again. Administrators who do not want progressive disclosure can enable **Display every tag immediately**. All tags are still emitted in the server-rendered HTML in every mode, so disabling JavaScript leaves the complete tag collection visible and usable rather than hiding content permanently.

Tag ordering is configurable as **Most used first** or **Alphabetical**. **Most used first** is the default. Usage is the total number of direct tag assignments to galleries plus direct tag assignments to photographs across the installation. Equal usage counts fall back to a natural case-insensitive alphabetical order and then a stable identifier tie-break. Direct gallery tags are sorted within their group and contained tags are sorted within their group; the groups are not mixed together.

Long tag collections can also use a responsive internal scrollbar. This option is enabled by default and defaults to five visible rows. The row threshold can be configured from 1 to 12 with a slider or exact number field. The browser measures the rows after the tags have actually wrapped at the current hero width, and it enables scrolling only when the measured row count exceeds the configured threshold. Resizing the page causes the measurement to be recalculated. Disabling the scrollbar option removes the height constraint and allows the hero to grow naturally.

20 Downloads, sharing, and transfer

Gallery downloads prepare authorized content as ZIP archives and stream the finished archive to the browser. A normal gallery download traverses the permitted gallery subtree; Smart Gallery downloads re-evaluate the saved query and current access rules; Picture Manager selection downloads package only the currently selected owned photographs; the administrator can also request an all-galleries archive. Archive entry names are normalized to prevent traversal and keep deterministic paths.

On a modern browser, a public gallery or Smart Gallery download first requests a private, bounded manifest. The browser then requests each authorized original through an independent media endpoint and assembles the ZIP locally, showing progress and allowing cancellation or retry without holding one long PHP archive request open. Every source request repeats the current gallery, Smart Gallery membership, visitor visibility, media-version, and path-containment checks; an image identifier alone never grants access. File-count, cumulative source-byte, duplicate-name, memory, and ZIP64 limits keep the workflow bounded. Direct links and no-JavaScript clients retain the existing server-side ZIP fallback.

Generated gallery/all/selection ZIPs use a content signature derived from the relevant gallery/image metadata so a still-current cached archive can be reused and a changed collection produces a new cache identity. This *content signature* is cache invalidation metadata, not a public HMAC download authorization token. Authorization is checked by the download controller before an archive is served.

20.1 Gallery-to-gallery migration

The gallery editor's API area supports both transfer directions between compatible PHP Gallery installations. **Export this gallery to another instance** treats the current gallery as the source and uses a gallery-scoped API key created for the receiving parent gallery. **Import into this gallery from another instance** treats the current gallery as the receiving parent and uses an API key created for the remote source gallery. The import creates a new child gallery below that parent; it never overwrites or moves the selected receiving gallery.

Include subgalleries is enabled by default. When selected, the versioned manifest describes the source gallery and its complete descendant hierarchy in parent-first order. The target recreates that hierarchy below the new imported root and preserves supported gallery/image metadata, translations, flight-map state, originals, existing thumbnails, and gallery branding assets. Turning the option off transfers only the selected source gallery. The current compatibility policy requires source and target to run exactly the same application version.

The receiving server creates a deterministic package plan bounded by its upload limits. Each original remains grouped with its already-generated thumbnails, while gallery-level assets can be packaged independently. ZIP entries use stable asset keys, store existing bytes without recompressing photographs, and are checked against the manifest's sizes and SHA-256 checksums before installation. Exact API-key scope, gallery-tree structure, target paths, database readiness, and asset completeness are validated at their mutation boundaries.

The administrator chooses a reconnect interval from 5 to 300 seconds. After a timeout, reload, or interrupted connection, the active side asks the durable target job which asset keys are already present and skips completed packages instead of blindly retransmitting them. Folder mappings and received assets are saved incrementally so the same transfer can resume safely. Completion remains a separate step and is refused until every declared asset is present. Cancelling the browser workflow stops further requests; credentials should still be revoked when a temporary migration

key is no longer needed.

20.2 Mobile WebDAV transfer

Galleries > Mobile uploads creates a WebDAV-compatible connection for one selected gallery. Record a recognizable label, choose the gallery, create the connection, and copy the generated server URL, username, and password into the mobile application. The password is shown only once and stored as a hash. Existing rows show their last-use time and can be deleted immediately from Admin.

Use the WebDAV target type in clients such as PhotoSync. Current server ingestion accepts JPG, PNG, GIF, and WebP, so enable HEIC-to-JPEG conversion in the client where available. WebDAV uploads do not bypass gallery ingestion rules: the credential is gallery-scoped, authentication is required on every write, the gallery/image schema must be known, and accepted media is then scanned, optionally auto-renamed, and processed through the ordinary thumbnail pipeline.

20.3 Public discovery and selected-photo tools

The public `/robots.txt` response describes crawler boundaries, while `/sitemap.xml` lists eligible public gallery and image URLs with image metadata and last-modified information. Unpublished, private, password-protected, age-restricted, or otherwise unauthorized content is not made public through these outputs. Canonical URLs, social metadata, and structured image data use the same access-aware public path model.

The Picture Manager is a logged-in administrator overlay on the public gallery view, not a visitor feature. An administrator can select photographs in the public grid, drag them onto a visible child gallery, move or copy them through the searchable destination picker, create a child gallery by copying the selection, and use the native device share sheet where supported. Native sharing falls back to the selected-photo ZIP when the browser cannot share the prepared files. The selection itself is only browser state and does not grant permission. Every mutation/ZIP request rechecks the normal logged-in account, CSRF token, source gallery, selected image ownership, and destination where applicable. Ordinary visitors do not receive these controls.

21 Maintenance, updates, and recovery

21.1 Migrations

Schema changes live in timestamped PHP files under `database/migrations/`. The migration runner prevalidates pending definitions, applies them in filename order, and records successful versions. New schema work receives a new migration file so existing installations retain a deterministic history.[23]

Database-readiness checks are remembered only for the current PHP request. This normally reduces repeated metadata queries, but a migration can change the answer while the same Admin, installer, updater, or command-line process is still running. The migration runner therefore clears those remembered answers after schema-changing statements and after successful repair callbacks. A post-migration check then reads the new database catalog instead of reusing an answer from before the migration. Ordinary data-only migration statements do not clear the readiness cache because they cannot create or remove a table, field, or index.

21.2 How the gallery checks database readiness

The shared readiness states and their owner actions are defined in Section 18.5. Maintenance uses the same contract when checking tables, fields, and indexes needed by password reset, NSFW Guard, upload automation, thumbnail metadata, and other schema-dependent features.

During an upgrade, **Missing** can be expected when new application files arrive before their migration has run. **Unknown** instead means PHP Gallery could not inspect the selected database reliably. For **Missing**, back up and apply the pending migration. For **Unknown**, check database connectivity, the selected schema, and metadata-inspection permissions before changing schema.

After repair, reload **System Health** and retest the affected feature. Keep any request reference with the time and application version when asking a hosting provider or maintainer for help.

21.3 Gallery trash and recovery

Open **Maintenance > Trash** to review recoverable gallery deletions. Each active row shows the original path, deletion time, gallery/photo counts, stored size, lifecycle status, and automatic-purge deadline when that policy is active. Problem entries remain visible for inspection; PHP Gallery does not guess that an ambiguous filesystem/database state is safe to destroy.

The main Trash switch controls future gallery deletes and is enabled by default. Turning it off does not hide or delete existing entries: restore and manual permanent deletion remain available. Automatic purge is a separate opt-in and is off by default. Its retention defaults to 30 days, its maintenance batch is bounded, and automatic deletion pauses whenever Trash is disabled. Enabling automatic purge after a pause gives current recoverable entries a fresh full retention window before scheduled deletion can begin.

Migrations `202609070001_gallery_trash_bin.php` and `202609070002_gallery_trash_state_machine.php` add the durable operation records. When Trash is enabled, deletion checks this schema before the first filesystem move or database-row deletion. Confirmed missing or unknown storage stops recoverable deletion and directs the administrator to apply/check migrations; it does not silently fall through to permanent deletion. Scheduled Site Maintenance reconciles stale

[23] Some migrations include repair callbacks in addition to SQL. Compatibility tests protect partial-update scenarios where an older runner encounters newer migration definitions.

in-progress operations conservatively and runs expired auto-purge work only when both Trash and automatic purge are enabled.

Trash payloads live in protected persistent `data/gallery-trash/` storage rather than the cache or live gallery tree. Deployment packages carry the directory's access-policy file but exclude stored trash contents. Include that runtime directory and the database in backup and restore planning; neither half alone is a complete recoverable entry.

21.4 Thumbnail maintenance

Maintenance tools inspect generated variants, identify missing or stale derivatives, and rebuild selected sizes. Browser-assisted rebuilding can prepare source chunks where configured. Background warm-up handles small public-rendering gaps with limited requests, while explicit Admin maintenance remains the primary large-scale repair path.

21.5 SEO request guard and public query hygiene

The maintenance area includes an optional public SEO request guard, enabled by default. For anonymous public GET requests it keeps an explicit allowlist of query parameters for each route. Known analytics parameters such as UTM fields and common ad-click identifiers are ignored because they do not change rendered content. Unexpected parameters are rejected before the normal public renderer with a 404, `noindex,nofollow`, and no-cache response so crawler-generated parameter spam does not multiply indexable gallery URLs.

Admin routes and machine endpoints such as setup, WebDAV, upload automation, gallery-migration receivers, and the maintenance cron endpoint are outside this public query guard. Signed-in administrators are also exempt so diagnostic/admin workflows are not blocked by public crawler rules. Rejected-query logging can be enabled independently and is sample-limited per UTC day to prevent a crawler flood from filling the Admin log. The same service supplies fallback canonical URLs when a public page has not already emitted its own canonical tag.

21.6 Database maintenance

Database inspection inventories schema objects, references, ownership categories, retention policy, and cleanup confidence. Logical cleanup uses allow-listed rules with deterministic selection, limited batches, and audit records. Schema repair provides a dry-run and idempotent migration path. `ANALYZETABLE` can refresh optimizer statistics for selected tables. Physical `OPTIMIZETABLE` is available only as a preview followed by separately confirmed selected-table operation; it is never an automatic cleanup step.[24]

21.7 Storage statistics and Complete report

Detailed **Storage statistics** are calculated only when an administrator requests them, so the normal dashboard does not perform a full derivative-filesystem scan on every visit. The Files view separates indexed source photographs from generated JPEG/WebP thumbnails and DNG browser-display masters, shows source file-type and size distributions, largest galleries and sources, unknown sizes, and scan warnings. A cached snapshot is marked stale when gallery data changes.

[24] Storage-engine allocation values such as data-free estimates describe engine state and do not guarantee a precise number of filesystem bytes returned after optimization.

Refreshing the snapshot scans expected generated files in small browser-driven batches. The same area links to database storage details and database maintenance.

The Database tab reads information-schema storage statistics separately from the filesystem scan. It distinguishes Gallery-owned tables from the whole selected database where possible and shows the largest tables rather than forcing a full table inspection into the ordinary dashboard request. The explicit **Recompute database statistics** action runs the database engine's statistics refresh for the available tables and reports partial failures in the Admin log instead of pretending the recompute was complete.

Maintenance > Complete report builds one self-contained HTML report covering storage, database structure and size, gallery structure, image and EXIF statistics, GPS overview, telemetry, operational logs, feature/settings state, and runtime diagnostics. Image and generated-media work is processed in bounded browser-driven batches so the report remains practical on shared hosting. GPS place summaries use local approximate clustering and offline reference data; probable simulator or game screenshots are excluded from real-world place frequency where the available evidence supports that classification.

The completed report is returned to the browser as a download and is not saved as a report file on the server. To keep a PDF copy, open the generated HTML and use the browser's Print or Save as PDF function; print styling is included in the report. Treat any exported report as administrative data because it can contain detailed operational and collection statistics.

21.8 Telemetry and privacy controls

The optional **Telemetry** area records privacy-controlled operational measurements and presents them as Admin reports. Available controls cover the master telemetry switch, public usage measurements, performance, cache and database events, honoring Do Not Track, excluding administrators, the maximum counted photo-view duration, and retention for raw, hourly, and daily data. Reports include measures such as anonymous sessions, page views, photograph opens and capped view time, client errors, measured media bytes, browser mix, and cache/database activity where those categories are enabled.

The telemetry subsystem is designed not to store raw IP addresses, raw browser user-agent strings, raw referrer URLs, names, email addresses, account identifiers, request bodies, or exact visitor locations. Exports contain aggregated anonymous telemetry and session hashes rather than raw visitor identifiers. Maintenance rolls up and purges telemetry according to the configured retention policy, and the Admin screen can export a standalone HTML report for offline review. Review the privacy controls before enabling collection on a public site.

21.9 Runtime diagnostics, integrity, and gallery benchmark

Runtime diagnostics provides a copyable environment and support view for PHP, database-readiness classes, hosting limits, GD, Imagick/ImageMagick, and relevant media-format support. It also exposes the DNG conversion policy described earlier. The readiness panels separate presentation/reporting dependencies, authentication/security dependencies, and mutation-sensitive dependencies so an administrator can distinguish an optional missing capability from a condition that must block a write.

The separate **Development diagnostics** switch is administrator-only. When enabled for a signed-in administrator it adds viewer instrumentation for preload, cache, memory, network, and frame-timing work in the public/lightbox/fullscreen experience. It does not turn those diagnostics on for ordinary anonymous visitors. Keep it off during normal ownership unless you are investigating

rendering or performance behavior; the gallery benchmark also depends on this development-diagnostics context.

The Admin **Integrity** workflow checks integrity-managed core files against the installed manifest and helps identify incomplete or unexpected deployment changes. It is a runtime verification tool; coordinated backups are still required because an integrity manifest does not contain user photographs or replace recovery media.

When development diagnostics are available, a signed-in administrator can start a **Public gallery load benchmark** for the current gallery. The browser loads the gallery several times in a hidden same-origin frame using an anonymous preview, combines PHP render counters with browser navigation timing, and then offers the benchmark log as JSON. Use the same gallery, presentation mode, and network/cache conditions when comparing two changes; the benchmark is most useful as a before/after measurement rather than as a universal score.

21.10 Scheduled site maintenance

Routine maintenance can run in bounded slices instead of one long request. The maintenance service chains safe web requests when traffic is available and can also be called from hosting cron with the CLI command shown in Admin. It can check thumbnail state, refresh metadata, process cleanup tasks, roll up telemetry, and archive eligible Admin-log days according to their configured policies. The hidden web-cron token is rotatable; rotating it invalidates previously published web-cron URLs. Maintenance is non-destructive by default and does not replace explicit confirmation for database optimization or other consequential actions.

21.11 Admin log history and archives

Everyday administrative work creates a useful history: uploads, gallery changes, security decisions, maintenance results, warnings, and other operational events can leave a record in the Admin log. That history is valuable when you want to understand what happened, when it happened, and which account or request was involved. It is also normal for the table to become large on an active installation. Version 0.97 therefore treats log care as its own explicit maintenance task rather than allowing generic database cleanup to remove audit history unexpectedly.

The Admin log screen can show individual entries or collapse repeated events into a summary. A group may represent repeated thumbnail warnings, upload results, or maintenance messages; opening it shows the individual records. Filters and workflow status controls help narrow operational review, while bounded exports preserve selected history without loading the entire table in one request. Large lists, raw-instance expansion, and exports are processed in limited portions, so requesting a complete history does not require the browser or PHP to hold the entire table in memory at once.

The owner can choose how long recent logs should remain in the live database. Choosing *Forever* disables automatic archival. Choosing a finite live period does not immediately throw older records away. The maintenance workflow first selects complete calendar days that are outside the live retention window, writes a protected archive containing both machine-readable JSON and a readable static HTML report, checks that the archive contains the expected number of records, and only then removes those same rows from the live table. The archive is stored under `data/admin-log-archives/`, which is application data rather than a public gallery area and carries an additional web-server protection rule.

Archive maintenance is resumable. It writes temporary files before promoting a completed archive, records a small state file, and uses a lock so two maintenance requests do not process the same

history simultaneously. If hosting limits, a connection interruption, or another problem stops the work, the Admin screen can show the incomplete state and the next maintenance attempt can verify, continue, or recover it. An archive is never considered a reason to delete live records merely because a file with a similar name exists; the row-count and archive metadata checks are part of the safety boundary.

Administrators should still include the database and the `data/admin-log-archives/` directory in coordinated backups. The live database contains recent operational history, while the archive directory contains older history selected by the explicit retention policy. Keeping both gives the owner a continuous record and makes a restored installation easier to investigate after an update, migration, or unexpected event.

21.12 Complete current feature reference

This reference is the manual's completeness checklist for Version 0.97. It lists the current product-level capabilities exposed by the public routes, Admin navigation, gallery/photo editors, maintenance tools, and optional integrations. Internal helper functions are not listed as separate features when they exist only to implement one row below.

Feature	Implemented behavior
Filesystem-first galleries	Nested physical folders mirrored by database records, unlimited practical hierarchy, discovery/import, manual creation, physical moves, clean public paths, sidecar metadata, and stable sibling/photo ordering.
Gallery metadata	Title, description, maintained translations, manual date/range, EXIF date suggestions, slug, folder name, parent, order, tags, cover/title picture, uploaded gallery thumbnail, branding, background, and presentation overrides.
Gallery access	Public, unpublished, private, password locking, expiring/non-expiring share links, inherited access evaluation, administrator access, and NSFW / 18+ confirmation boundaries.
Bulk gallery administration	Branch-aware deletion, visibility updates, EXIF/GPS map override changes, voting changes, filename-display changes, Picture Game changes, path regeneration, thumbnail/scanning operations, and sibling reorder/date-sort workflows.
Smart Galleries	Saved nested Boolean queries over physical galleries, tags, dates, EXIF/GPS, text, AI metadata, duplicate state, file properties, and private editorial rating; public/unlisted/root/attached placement; per-parent above/below ordering; cycle protection; independent presentation overrides; complete paged lightbox traversal; bounded ZIP download.
Photograph editor	Title/description/translations, visibility, NSFW, sort order, private editorial rating, tags, per-photo thumbnail bounds, EXIF/GPS review, internal AI metadata review, and OpenAI draft tools.
Bulk photograph administration	Move to existing/new gallery, deletion, cover assignment, visibility, NSFW, thumbnail creation, and drag/filename ordering.

Feature	Implemented behavior
Tags	Shared gallery/photo tags, normalized reusable tag names/slugs, public descriptions, usage counts, weighted editor suggestions, Admin rename/edit/delete, public tag landing pages, tag-specific grid/card settings, hero tag ordering/disclosure/scrolling.
Public home and gallery pages	Hierarchical cards, breadcrumbs, covers/collages, pagination, direct photograph grids, gallery counts, dates, descriptions, branding/backgrounds, favorite header shortcuts, contextual logged-in Admin edit/remove/reorder controls, floating back-to-top assistance for long listings, responsive public paths, and query-string fallback when rewriting is disabled.
Public search	Optional site/branch search across gallery/photo titles, descriptions, filenames, tag metadata, maintained-language text, and available internal AI searchable text; minimum query length and bounded result count; access/listing policy applied before results are returned.
Robots, sitemap, and SEO	Access-aware `robots.txt`, image-aware `sitemap.xml`, canonical/social/structured metadata, plus the route-specific query-string guard described above.
Public media authorization	Dedicated original/thumbnail/cover/branding/background/favicon routes re-evaluate gallery/image visibility and protected access instead of exposing arbitrary filesystem paths.
Thumbnail generation	Responsive size variants, JPEG/WebP compatibility policy, responsive and progressive selected-gallery renderers, lazy/near-viewport behavior, per-gallery/photo size bounds, DNG display masters, metadata reconciliation, Admin/browser rebuild workflows, and signed access-checked low-budget public fallback warmup.
Lightbox	Authorized asynchronous metadata, keyboard/touch navigation, slideshow/fullscreen, single-image/picture-strip/3D-carousel browsing modes, EXIF/map overlay, voting state, 100–400% zoom with pan/pinch/cursor anchoring, immediate original-quality promotion when zooming, and Shift+Arrow jumps of 10 photographs.
EXIF and GPS	Camera/lens/exposure/date/GPS extraction, global default with per-gallery inherit/force-on/force-off, photo pins, gallery map data, date suggestions, duplicate evidence, and privacy-sensitive public suppression.
Voting	Per-image reversible positive vote with score synchronization, anonymous visitor hash or logged-in ownership, rate limiting for new likes, and per-gallery enable/disable.
Picture Game	Optional branch-level pair comparison using eligible public images, persistent pair/winner state, leaderboard/result aggregation, and schema-safe refusal when storage is unknown.

Feature	Implemented behavior
Picture Manager	Logged-in public-view administrative selection overlay with checkmark/range/toggle/select-all controls, move/copy/create-child actions, drag-to-visible-subgallery movement, native multi-file Web Share where supported, and selected-photo ZIP fallback/download.
Downloads	Authorized gallery ZIPs, Smart Gallery ZIPs, selected-photo ZIPs, administrator all-gallery ZIP, normalized archive paths, cache content signatures, bounded Smart Gallery source-count/byte limits, and safe cache cleanup.
Browser uploads	Existing/new-gallery upload flows, explicit browser or server processing choice, complete prepared-thumbnail validation, bounded workers/batches with singleton oversize packages below the hard PHP limit, format-policy hinting, local ZIP import, and post-scan automatic renaming.
Filesystem discovery	Session-backed bounded folder scan; skips symlink/hidden/generated directories; imports image-bearing hierarchy in place; can move unmanaged photos into an existing gallery or delete safe unmanaged candidate trees; reports metadata-only sidecars separately.
Filesystem metadata sidecars	Optional per-gallery `gallery.json` written from editable gallery state and read during discovery/parent repair for supported metadata; useful for filesystem portability but not a replacement for database backup.
Upload API	Gallery-scoped one-time-visible API keys, central API manager, key revocation, shared ingestion pipeline, and authenticated automation endpoint.
Windows companion	Watch-folder uploads, manual bulk upload, optional local responsive-thumbnail generation, parallel/pipelined work, retry/state files, system tray, optional SimConnect camera coordinates, confirmed-success source deletion, and optional local AI metadata worker.
Mobile WebDAV	Gallery-scoped one-time password credentials, Basic authentication, WebDAV discovery/write methods required by mobile transfer clients, JPG/PNG/GIF/WebP ingestion, last-use tracking, and revocation.
Media Renamer	Dry-run physical filename plan, default `\{gallery_path\}_\{seq4\}` and documented wildcard placeholders, collision/safety handling, gallery-order context, coordinated database/public-path/derivative/cache updates, bounded site-wide application, and post-upload auto-rename integration.
Metadata Organizer	Preview-only EXIF date grouping with min/max date filters, ISO `YYYY-MM-DD` child folders, reuse of matching direct date children, confirmed physical moves with derivatives, and bounded apply batches; location grouping remains planned rather than active.

Feature	Implemented behavior
Duplicate Photo Detector	Exact SHA-256 and metadata-supported possible pairs, context links, deletion workflow, per-admin reviewed-pair/exact-gallery ledger, and persistent review state.
OpenAI text assistance	Per-account encrypted API key/model settings, optional explicit thumbnail-consent switch, gallery/photo description drafting, text cleanup, context summary, visible-content description from small generated thumbnails, translation suggestions, and manual-review-only insertion.
Internal AI metadata	Separate searchable machine metadata and queue tables, authenticated Windows worker claims/results, read-only photo-editor inspection, Smart Gallery/search evidence, model-generation tracking, and forced branch reprocessing.
Flight maps and SimBrief	Manual route text, explicit coordinate syntax, SimBrief latest-OFP fetch, editable generated Markdown, saved OFP data and coordinates, and public rendering from stored points only.
Navigation data	OurAirports local database import, bundled offline CSV, cached provider points, optional Navigraph OAuth/package/AIRAC integration, offline-first resolver, and Admin lookup tester.
Gallery migration	Same-version source-push/target-pull API transfer, versioned manifests, metadata/asset checksums, one-asset bounded requests, durable target status, reconnect/resume, and separate completion.
Theme and layout	Direct palette colors, font mode, rounded corners, page width, gallery/card grids, pagination, card layout, count badge, public thumbnail renderer, lightbox default, GPS-pin presentation, three favorite/header shortcuts, and live preview. There is no separate light/dark-mode switch in the current Theme UI.
Branding and browser identity	Site header banner, separator dimensions/stretching, optimized background and opacity, per-gallery overrides, public cover assets, square-cropped favicon with 32/48/180-pixel generated variants, and custom CSS presets/editor.
Localization	English canonical fallback plus maintained selectable Czech, German, and Swedish UI packs, per-viewer browser preference, flag/native-name selector designs, shared browser/server catalogs, pack diagnostics/editor/import/export, and per-gallery/photo maintained-language content.
Feature switches	Global route-aware switches for public search, lightbox modes, Picture Manager, image voting, Picture Game, downloads, viewer accounts and private collections, gallery maps, flight maps, navigation data, SimBrief, OpenAI, local AI metadata, upload API, mobile WebDAV, gallery migration, Media Renamer, and telemetry.
Central Settings	Searchable categorized registry, keyboard-driven Spotlight navigation, canonical save delegation for approved global values, specialist deep links, inheritance-aware ownership, default/source labels, and secret redaction.

Feature	Implemented behavior
Admin gallery workspace	Visibility filter with hidden-row deselection, persistent per-branch collapse state, Collapse/Expand all, hierarchy reordering, visible-page public reorder tools, side-panel editors, and contextual public-page edit/remove controls for signed-in administrators.
Administrator authentication	Username-or-recovery-email login, CSRF/session protection, hashed rate-limit subjects, optional durable login token, password reset by PHP mail/SMTP, account/profile changes protected by current-password checks, and optional Google sign-in that must first be linked from an authenticated password session and requires the current password to disconnect.
Viewer accounts, lifecycle, favourites, private collections, and unlisted sharing	Administrator direct viewer creation/deletion plus invite-only registration, Admin suspend/restore/sign-out-everywhere controls with central viewer-authority revocation, forced replacement of Admin-provisioned temporary passwords before normal viewer authority, scanner-safe email verification and activation, separate viewer login/logout, dedicated rotating remember-me credential, generic scanner-safe password reset, recent password reauthentication for sensitive account actions, password/email change, viewer self-deletion, private no-store account/favourite/collection/lifecycle responses, favourite controls on authorized cards/lightbox images, viewer-owned ordered collections with live source authorization, one revocable 30-day unlisted read-only collection link with show-once secret and clean redirect, recipient-context source ACL intersection without Admin bypass, and strict separation from gallery authorization; no open signup, public collection discovery/profiles, recipient collaboration, comments, or uploads.
Database readiness/System Health	Tri-state schema inspection separating Available, Missing, Unknown, and Disabled where applicable; distinct security/authentication, mutation-sensitive, and optional presentation/report capability panels; writes fail closed on unknown required schema.
Database maintenance	Explicit schema/migration/code-reference inventory, storage information, dry-run cleanup candidates, bounded logical cleanup, dedicated legacy repair readiness, selected ANALYZE, selected-table optimization preview, and separately confirmed OPTIMIZE TABLE.
Storage statistics	On-demand source/generated-media scan, file-type/size distribution, largest galleries/files, DNG master and thumbnail accounting, stale snapshot handling, separate database/table usage view, explicit database-statistics recompute, database-maintenance links, and bounded refresh batches.
Complete report	Self-contained HTML export covering storage, database, gallery/image/EXIF/GPS statistics, feature/settings state, telemetry, logs, and runtime diagnostics, with bounded image/generated-media processing.

Feature	Implemented behavior
Telemetry	Optional privacy-controlled anonymous usage/performance/cache/database measurements, DNT respect, administrator exclusion, bounded view time, raw/hourly/daily retention, rollups/purge, Admin dashboard, and HTML export.
Admin logs	Structured events, filters/status, repeated-event grouping with member expansion, bounded exports/ZIP export, configurable live retention, protected daily JSON+HTML archives, resume/recovery state, and archive download/view tools.
Scheduled site maintenance	Daily UTC window, optional request-triggered continuation, bounded image batch/time budget, CLI/web-cron support, run-one-slice action, interrupted-state reset, rotatable hidden web-cron token, thumbnail/metadata/cleanup/telemetry/log-archive tasks.
Runtime diagnostics	PHP/hosting/database readiness, GD/Imagick/ImageMagick/format support, DNG policy, copyable support summary, administrator-only development diagnostics for viewer preload/cache/memory/network/frame timing, and development-only benchmark entry points.
Integrity	Installed core-manifest hash verification for managed application files plus release-time manifest generation/validation; separate from user-data backup.
Gallery benchmark	Development-only authenticated browser benchmark that uses anonymous preview loads, same-origin hidden frames, server render counters and navigation timing, then exports JSON for before/after comparisons.
Updater	Stable/beta discovery, cached patch notes, resumable durable jobs, Range download resume, archive/manifest verification, staging, rollback snapshot, activation, migrations, cleanup, locks/stale-lock recovery, retry/cancel before activation, file rollback after activation, stable restore/clean reinstall/beta install, CLI continuation, and emergency `reset.php`.
Deployment helpers	Windows/Linux deploy scripts, release metadata, core-manifest generation and package validation, with deployment designed not to make a locally installed PHP executable an unconditional runtime requirement for the gallery itself.

21.13 Application updates

The updater checks configured channels, validates packages, stages content, verifies required runtime files and sizes, preserves overwritten files in recovery storage, and activates the update. Rate-limit handling and retry state prevent repeated remote requests during provider wait periods. A resumable job card appears in both **Status** and **Advanced tools**, so a beta install, restore, or reinstall continues to show its progress where it was started. The page may be closed and reopened without discarding completed checkpoints.

The Admin update area also shows cached release information and versioned patch notes for the available channel, so an owner can review a release before starting its job. Stable and beta channels remain separate choices; the Beta label belongs to the release choice in the Admin

interface and does not change the updater's durable job or manifest terminology.

Temporary download or hosting failures may be retried. A package whose files do not match its own integrity manifest is different: downloading the same published build again cannot change those bytes. PHP Gallery therefore asks the administrator to cancel that prepared job and choose a newer release or beta code instead of offering an endless retry loop.

Stable discovery and package work are separate bounded operations. A discovery request may create a durable background job, while a later safe request or the CLI runner advances a short slice through download, archive validation, extraction, manifest verification, staging, rollback backup, activation, migrations, and cleanup. Range requests can resume interrupted downloads, worker locks prevent concurrent advancement, and a stale lock can be recovered after its owner expires. On idle shared hosting, schedule `phpscripts/application_update.php` from cron, for example every five minutes. Each invocation performs bounded discovery or advances existing background work; correctness does not depend on `set_time_limit()` or `ignore_user_abort()`.

Current releases also reconcile the small set of application-owned server-policy files during update preparation and activation. This protects installations whose older updater skipped a policy file: the validated release copy is compared and published atomically, rollback metadata is extended before replacement, and a migration-backed bootstrap path can repair an already activated older tree. The reconciliation is limited to known policy paths and does not alter gallery media or user configuration.

Before activation, an administrator can cancel or retry a recoverable job. After activation, the Admin recovery action can restore managed application files from the saved rollback snapshot, but file rollback does not reverse database migrations. Stable restore, clean reinstall, and beta installation remain resumable jobs with their own package validation. The standalone `reset.php` emergency path is a separate recovery tool for returning to the stable branch when the normal Admin route cannot be used; it still requires the same careful backup and integrity review.

During the short activation critical section, a dependency-free early runtime gate prevents new ordinary requests from loading a partially replaced application. Such requests receive a private, non-cacheable `503ServiceUnavailable` response until activation completion is durably recorded. The authenticated update recovery/status path remains available, so an interrupted activation can continue. Corrupt or incomplete gate state fails closed instead of presenting the installation as healthy.

The same early runtime boundary converts uncaught and catchable fatal PHP failures into bounded `500` responses when PHP can still control the response. Production hosting must keep `display_errors` disabled; otherwise the PHP runtime itself may print warning or fatal details before application-level handling can format a safe response.

> **Important.** Run a coordinated backup before an application update. Keep the backup available until runtime checks, migrations, public galleries, protected access, uploads, and Admin operations have been verified.

21.14 Integrity manifest

`app/core-manifest.json` records the application version and hashes for integrity-managed core files. The local generator under `scripts/generate_manifest.php` calculates final hashes after runtime changes. Documentation and user-data inclusion follow the generator's established policy.

22 Optional background: how PHP Gallery works

The following pages describe the request path used by the application. They are mainly relevant to maintenance and development; ordinary gallery administration does not require them.

22.1 Bootstrap, routing, and dispatch

Ordinary requests enter through `public/index.php`; the repository-root `index.php` is a supported compatibility entry point for other hosting layouts. The public entry loads `app/early\_runtime.php` before `app/bootstrap.php`, then both layouts reach the same normal request pipeline.

Bootstrap reads local configuration and establishes the request context: database access where required, language, session state, security prerequisites, and shared runtime services. Routing then translates the incoming URL or query-style fallback into an internal route plus parameters such as a gallery path, page number, media size, or share token. Dispatch calls the controller registered for that route.

The ordering is deliberate. A request must be classified before code decides how to serve it, and access checks must run before protected output is assembled. A direct-looking image or thumbnail URL is therefore still an application request; the media controller authorizes it before returning bytes.

```
browser request
  -> index.php or public/index.php
  -> app/bootstrap.php
  -> cms_run()
  -> cms_route_from_request()
  -> controller function
  -> service functions
  -> HTML, JSON, media, archive, or redirect response
```

22.2 Controllers

Controllers own HTTP-level coordination. They receive a resolved route and request context, validate the operation, call the services needed for that operation, and choose the response type. A public gallery route normally returns HTML; an Admin side-panel action may return JSON or an HTML fragment; a media route returns an authorized file; a download route can stream an archive.

Public gallery rendering begins in `app/controllers/public_gallery.php`, with related responsibilities split into focused controller files. Uploads, maintenance, updates, downloads, media access, and Admin features have their own entry points, while shared domain rules remain in services.

Controllers are not an authorization shortcut. Thumbnail, map, lightbox, search, download, and direct-media routes repeat the relevant access decision instead of assuming that a visitor previously opened the gallery page successfully.

22.3 Services

Reusable application rules live in `app/services/`. Services resolve galleries and images, evaluate access, normalize settings, manage uploads and mutations, prepare thumbnail/media manifests,

perform searches, maintain metadata, and coordinate other domain work. A controller should be able to ask for one of these operations without reimplementing its rules.

The separation matters because the same photograph can be reached from several routes. Access policy, thumbnail selection, or deletion semantics should not depend on whether the request came from a gallery card, search result, lightbox, Admin tool, or direct link. Source photographs remain in the gallery tree; services coordinate the database records and generated derivatives that describe or support them.

22.4 Views and browser modules

Views render prepared data into HTML. They are responsible for structure and presentation, not for deciding whether a visitor is authorized or how a thumbnail is generated. Server output includes semantic links, forms, titles, alternative text, and initial media candidates before JavaScript enhancement begins.

Browser modules add lightbox behavior, search suggestions, voting, upload progress, drag-and-drop actions, thumbnail upgrades, diagnostics, and the Admin side panel. The code remains framework-free. Public pages therefore retain a useful server-rendered baseline, while authenticated workflows can add richer in-place interactions.

The Admin side panel is dynamic: its fragment can be replaced without navigating away from the page. Modules that own panel controls consequently use delegated or lifecycle-aware event binding so a freshly rendered copy continues to work and an old fragment no longer owns listeners.

22.5 Public selected-gallery render pipeline

A selected-gallery request first resolves the gallery and its effective access policy, including inherited protection, password/share state, publication state, and age restrictions. Only permitted content proceeds to rendering.

The controller then loads the direct child galleries and the current page of photographs, together with the metadata needed for titles, descriptions, tags, dates, votes, ordering, and navigation. Thumbnail metadata is prepared in batches, and request-local media manifests describe the authorized candidates available for each visible image.

The view receives that prepared model and renders the page title, branding, navigation, breadcrumbs, cards, pagination, lightbox configuration, and footer assets. Browser modules enhance the result after delivery. Access remains a server responsibility; JavaScript can make an interaction faster or more convenient, but it does not turn an unauthorized candidate into an authorized one.

This server-first split also preserves useful behavior when scripts are blocked or late. Direct links, semantic content, and the initial image choices are already present in the HTML, while JavaScript handles the features that genuinely need an interactive client.

23 Optional background: what the database remembers

The database stores application state that does not belong in image filenames: descriptions, privacy, ordering, tags, votes, dates, public addresses, accounts, and maintenance history. The overview below is enough for backup planning and hosting discussions; column-level definitions remain in the database reference [3].

23.1 Principal domains

Domain	Persistent responsibility
Users and authentication	Administrator identity, password hashes, recovery email, external identity links, reset tokens, and scoped credentials.
Galleries	Folder path, hierarchy, public path, metadata, visibility, access, branding, presentation overrides, and operational ordering.
Images	Gallery ownership, relative path, filename, metadata, dimensions, visibility, checksum, EXIF data, ordering, and derived state.
Tags	Reusable tag identity plus gallery and image relationships.
Votes	Gallery and image scoring records with viewer association.
Thumbnails	Valid generated variant inventory, dimensions, format, status, and derivative version.
Settings	Scalar application, theme, feature, update, maintenance, and rendering values.
Audit and telemetry	Administrator events, operational diagnostics, privacy-controlled measurements, and rollups.
Duplicate ledger	Per-administrator canonical pair decisions and exact-gallery suppression rules.
Jobs and tokens	Bounded browser, detector, migration, WebDAV, reset, sharing, and maintenance state where persistence is required.

23.2 Relations and cascades

Foreign keys express ownership where schema evolution and compatibility permit it. Cascades remove dependent workflow records when their parent identity disappears. Content deletion continues through application services so filesystem and database effects remain coordinated. Unique constraints encode identities such as canonical duplicate pairs, exact administrator-gallery rules, slugs, tokens, and metadata variants.

23.3 Application settings

The `app_settings` table stores key-value configuration. `app_setting()` reads values, `set_app_setting()` persists normalized values, and focused services interpret domain-specific settings. Public language configuration uses `public_language` for the site default, `public_language_selector_enabled` for the default-on viewer override feature, and `public_language_selector_languages` for its ordered non-empty language list. The public thumbnail renderer uses `public_thumbnail_rendering_mode` with `responsive` and `progressive` values.

24 Optional reference for developers

This reference is for people changing PHP Gallery itself. Owners who only need publication and maintenance checks can continue with Section 25.

24.1 Repository organization

Location	Responsibility
`app/controllers/`	HTTP controllers and response coordination.
`app/services/`	Reusable domain and infrastructure services.
`app/views/`	Layout and reusable server-rendered view helpers.
`app/lang/`	Server and browser translation catalogs.
`database/migrations/`	Timestamped schema evolution.
`public/assets/`	Framework-free JavaScript, CSS, and public assets.
`scripts/`	Migration, integrity, deployment, and maintenance utilities.
`tests/`	Standalone PHP and focused JavaScript tests.
`winapp/`	Windows-specific tools and integrations.

24.2 Coding conventions

New PHP files use strict types and four-space indentation. Controllers coordinate request behavior, while services own reusable logic. New JavaScript and CSS remain framework-free and belong under public assets. Migrations use timestamp-prefixed filenames. Existing explanatory comments, docstrings, and PHPDoc receive preservation and correction as behavior evolves.

Atomic modules should own one cohesive responsibility. Shared contracts deserve normalized inputs, explicit outputs, and focused tests. Dynamic Admin and public fragments require lifecycle-safe setup and teardown.

24.3 Adding a feature

A typical feature change follows this sequence:

1. Trace current implementation and identify the owning controller, services, views, browser modules, persistence, and tests.
2. Define access, CSRF, scope, fallback, migration, and no-JavaScript contracts.
3. Add focused services and route integration.
4. Add append-only migration work where persistent schema changes are required.
5. Add translated interface strings.
6. Add focused automated tests and documented manual verification.
7. Update architecture, code map, database, testing, and user guidance according to their ownership.

8. Regenerate integrity metadata after final runtime changes.
9. Review the complete diff and run the relevant local test suite.

25 Checking your gallery before publication

Testing for a gallery owner means walking through the site as the intended audience. A technically healthy page can still contain a private location, an unclear title, an incorrect date, an accidental download permission, or an awkward mobile layout. A short human review catches these presentation and privacy concerns.

25.1 A visitor-centered publication check

Open a private browser window and follow the same link you plan to share. Check:

1. The title and first paragraph explain the collection.
2. The chosen cover remains clear at a small size.
3. Child galleries appear in a sensible order.
4. Photograph titles and descriptions match the visible subjects.
5. Private photographs and galleries stay hidden.
6. Password or share access works from a fresh session.
7. GPS maps reveal only intended locations.
8. The first page loads comfortably on a phone.
9. The large viewer, forward and backward controls, and close action feel natural.
10. Search, tags, voting, and downloads behave according to the gallery's purpose.

Ask another person to try the gallery without instructions. Their first click and first question often reveal navigation or wording that felt obvious only to the owner.

25.2 Automated checks for software maintainers

Developers can run the project's executable test scripts after changing application code. The aggregate runner covers the project suite, while focused scripts verify particular features.

```
php tests/run.php
php tests/public_thumbnail_rendering_model_test.php
php tests/public_thumbnail_markup_test.php
php tests/duplicate_photo_detector_test.php
php tests/duplicate_photo_ledger_test.php
php tests/migration_consistency_test.php
node tests/progressive_thumbnail_renderer_test.mjs
```

25.3 Manual browser verification

25.3.1 Network and interaction checks

Browser-dependent software checks use representative data, anonymous and authenticated sessions, an empty cache, mobile and desktop viewports, JavaScript-disabled operation where relevant, reduced-motion preferences, and network throttling.

For public thumbnail rendering, inspect the Elements and Network panels. Responsive mode should expose its complete active candidate set. Progressive mode should expose a small active candidate and inert larger candidates before activation, then perform at most the documented concurrent upgrades for relevant cards. Check layout stability, cache reuse, error fallback, lightbox interaction, maps, voting, protected media, and direct links.

25.4 Release hygiene

A release review confirms runtime version consistency, ordered migration compatibility, manifest hashes, documentation, translations, browser cache-busting, tests, package exclusions, and user-data safety. The maintainer starts from a clean release branch, compares it with the exact previous release tag, reviews every intervening commit and path, and records anything intentionally excluded. The runtime version in `app/bootstrap.php`, the `release-metadata.json` key and tag, newest `PATCH_NOTES.md` heading, documentation markers, intended archive name, and final Git tag must all agree.

New migrations are reviewed in timestamp order and exercised through migration-consistency and legacy-runner tests. The maintainer rebuilds this PDF with the complete `docs/LATEX_BUILD.md` sequence and inspects its title page, release overview, contents, bookmarks, index, and changed feature sections. Every changed PHP file receives `php -l`; changed JavaScript and Node fixtures receive syntax checks; focused feature tests, translation consistency, documentation coverage, the complete PHP suite, and `git diff --check` must pass.

Only after the final source and documentation edit does the maintainer run `phpscripts/generate_manifest.php` and its `--check` mode. The deployment helper then creates the requested local folder or ZIP; it only packages files and does not execute PHP or repair a stale manifest. Inspect the archive listing and confirm it contains the current PDF, patch notes, release metadata, migrations, and `app/core-manifest.json`, while protected configuration, caches, logs, temporary files, local tools, deploy output, and user media follow the packaging policy. Recheck the manifest after packaging, create the release commit and annotated `v_<version>` tag from the verified tree, and smoke-test an updater upgrade from the previous stable release. Missing local dependencies must be reported with exact blocked commands and environment details rather than calling the release fully verified.

26 Making the gallery feel fast

Perceived speed is not one number. Visitors notice when the page first appears, when the first photographs become recognizable, whether scrolling stays responsive, how quickly controls react, and how long a larger image takes after they ask for it. Connection quality, server distance, page size, thumbnail policy, branding assets, and the selected renderer all contribute.

26.1 Practical ways to improve speed

- Keep the number of cards per page reasonable for the target audience and hosting plan.
- Generate the configured thumbnail sizes in advance for large public collections.
- Avoid unnecessarily large hero, background, logo, and other theme assets.
- Prefer the responsive thumbnail renderer when predictability and no-JavaScript behavior matter most; use the progressive renderer when its staged sharpening suits the collection and browser mix.
- Place the gallery on hosting geographically close to its main audience where practical.
- Test with an empty cache. A warm developer browser can hide expensive first-visit transfers.
- Check PHP/database latency separately from image-transfer time before assuming that every slow gallery is a thumbnail problem.

26.2 Slow-connection test

Open the gallery in a private browser window, select a representative mobile viewport, enable network throttling in developer tools, and reload with an empty cache. Watch the order in which the page, covers, visible cards, and later image upgrades arrive. Repeat with the renderer or page-size setting you intend to compare; otherwise cached files make the comparison unreliable.

26.3 Technical measurements for maintainers

Public render profiling and browser diagnostics are useful when a subjective report needs numbers. Record at least:

- time to first byte and server-side render duration;
- HTML transfer size;
- request count and bytes transferred for initial thumbnails;
- time until the largest visible content has settled;
- layout shifts while cards and media load;
- main-thread work attributable to browser modules;
- number and size of progressive candidate upgrades;
- database-query totals from the development profiler where available;
- image decode/memory pressure when testing very large originals or deep lightbox zoom.

The development profiler and captured benchmark records are intended for before/after comparisons across renderer modes and gallery layouts. Use representative high-density screens and constrained connections rather than relying only on a fast desktop with a warm cache.

27 Troubleshooting

27.1 Application does not start

Confirm the PHP version, required extensions, configuration location, database connectivity, writable paths, and web-root routing. Review PHP and web-server logs. Run syntax checks on recently changed PHP files and inspect pending migrations.

27.2 Gallery or images are missing

Confirm filesystem paths, gallery discovery state, database records, visibility, parent access rules, password unlock state, NSFW policy, supported file type, and image relative path. Test as both administrator and anonymous visitor.

27.3 Thumbnails are missing or soft

Review thumbnail metadata and maintenance reports. Confirm generated sizes, format support, configured bounds, source geometry, derivative status, and public endpoint authorization. In responsive mode, inspect the browser-selected candidate and `sizes`. In progressive mode, inspect the small candidate, queue state, selected replacement, and decode result.

27.4 Admin actions leave the side panel

Confirm the matching JavaScript module loaded, delegated handlers recognized the form, AJAX headers and response contracts are correct, the returned fragment contains expected lifecycle markers, and generic Admin form handlers exclude feature-owned forms. Direct POST behavior can still provide the documented fallback route.

27.5 Migration fails

Record the exact migration filename, database error, server version, existing schema state, and migration audit rows. Use available dry-run or inspection tools. Preserve the database before repair and follow the migration's idempotent compatibility path.

27.6 PDF manual compilation fails

Run the commands in `docs/LATEX_BUILD.md`. Confirm `pdflatex` and `makeindex` are installed. A first MiKTeX run may request common packages. Delete ignored intermediate build files after a failed package transition and repeat the documented sequence.

28 Backup and recovery checklist

1. Export the complete application database.
2. Copy `galleries/` with source files and gallery-owned assets.
3. Copy local configuration and relevant installation secrets securely.
4. Preserve custom theme assets and custom CSS.
5. Record the application version and pending migration state.
6. Store backups outside the active web tree.
7. Test restoration periodically in an isolated environment.
8. Verify public routes, protected access, Admin login, thumbnails, uploads, and migrations after restoration.

Updater-created backups preserve overwritten application files for update recovery. A complete operational backup also includes the database, gallery tree, local configuration, and installation-owned assets.

29 Glossary

AJAX Browser request used to update a page region while preserving the surrounding interface.

Branch scope A gallery and its descendants, when the selected feature defines recursive scope.

Canonical pair Two image identifiers stored in a stable low-to-high order.

CSRF token Session-bound value used to validate intentional state-changing form submissions.

Derivative Generated media representation such as a JPEG or WebP thumbnail.

EXIF Camera and capture metadata embedded in supported image files.

Integrity manifest Versioned list of core paths and expected hashes.

Lightbox Fullscreen or overlay photo viewer with navigation and related controls.

Media manifest Request-local rendering data prepared from thumbnail metadata for visible images.

NSFW guard Age and gallery/image policy controlling access to restricted media.

Progressive sharpening Small-first thumbnail pipeline that activates a decoded larger candidate later.

Responsive selection Browser-native candidate selection from an immediately active source set.

Selected gallery Gallery represented by the current public route.

Side panel Admin right-side contextual interface loaded and refreshed through fragment requests.

Thumbnail bounds Gallery-aware minimum and maximum generated sizes eligible for selection.

30 References

References

[1] PHP Gallery project, *README.md*, local product overview, feature guide, requirements, and installation documentation, version 0.97.

[2] PHP Gallery project, *ARCHITECTURE.md*, local application architecture and subsystem lifecycle reference, version 0.97.

[3] PHP Gallery project, *DATABASE.md*, local migration and persistent schema reference, version 0.97.

[4] PHP Gallery project, *CODEMAP.md*, local source ownership and maintenance map, version 0.97.

[5] PHP Gallery project, *docs/ADMIN_SETTINGS_INVENTORY.md*, canonical global Settings ownership, fallback, sensitivity, and migration inventory, version 0.97.

[6] PHP Gallery project, *TESTING.md*, local automated and manual verification guide, version 0.97.

[7] PHP Gallery project, *AGENTS.md*, local repository contribution and security guidelines.

[8] The PHP Documentation Group, *PHP Manual*, https://www.php.net/manual/en/.

[9] Oracle Corporation, *MySQL Reference Manual*, https://dev.mysql.com/doc/.

[10] MariaDB Foundation, *MariaDB Server Documentation*, https://mariadb.com/docs/server/.

[11] WHATWG, *HTML Living Standard: Images*, https://html.spec.whatwg.org/multipage/images.html.

[12] OWASP Foundation, *Web Application Security Guidance*, https://owasp.org/.

Index